

Chapter

02

컴퓨터 시스템과 운영체제

1. 컴퓨터 시스템과 하드웨어
2. 컴퓨터 시스템과 운영체제
3. 커널과 시스템 호출
4. 운영체제와 인터럽트



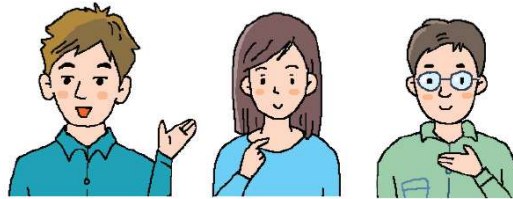
강의 목표

1. 컴퓨터 시스템에서 하드웨어의 구조와 작동 원리의 핵심을 이해한다.
2. 컴퓨터 시스템에서 운영체제의 위치와 역할을 알고, 그에 따른 운영체제의 전체 기능을 이해한다.
3. 운영체제가 응용프로그램과 하드웨어 사이의 중개자로서 제공하는 시스템 호출과 인터럽트에 대해 자세히 이해한다.
4. 응용프로그램이 실행되는 2가지 모드, 사용자 모드와 커널 모드에 대해 이해하고, 이들을 두는 목적 등을 이해한다.
5. 운영체제는 메모리 공간을 사용자 공간과 커널 공간으로 나누어 활용함을 이해한다.
6. 운영체제 커널의 개념과 실체에 대해 충분히 이해한다.

1. 컴퓨터 시스템과 하드웨어

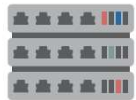
컴퓨터 시스템을 구성하는 계층

4



응용소프트웨어

운영체제



네트워크
장치



키보드



마우스

CPU, 캐시 메모리,
RAM 메모리, 디스크,
네트워크, MMU, DMAC 등

컴퓨터 하드웨어



디스플레이



하드
디스크



프린터

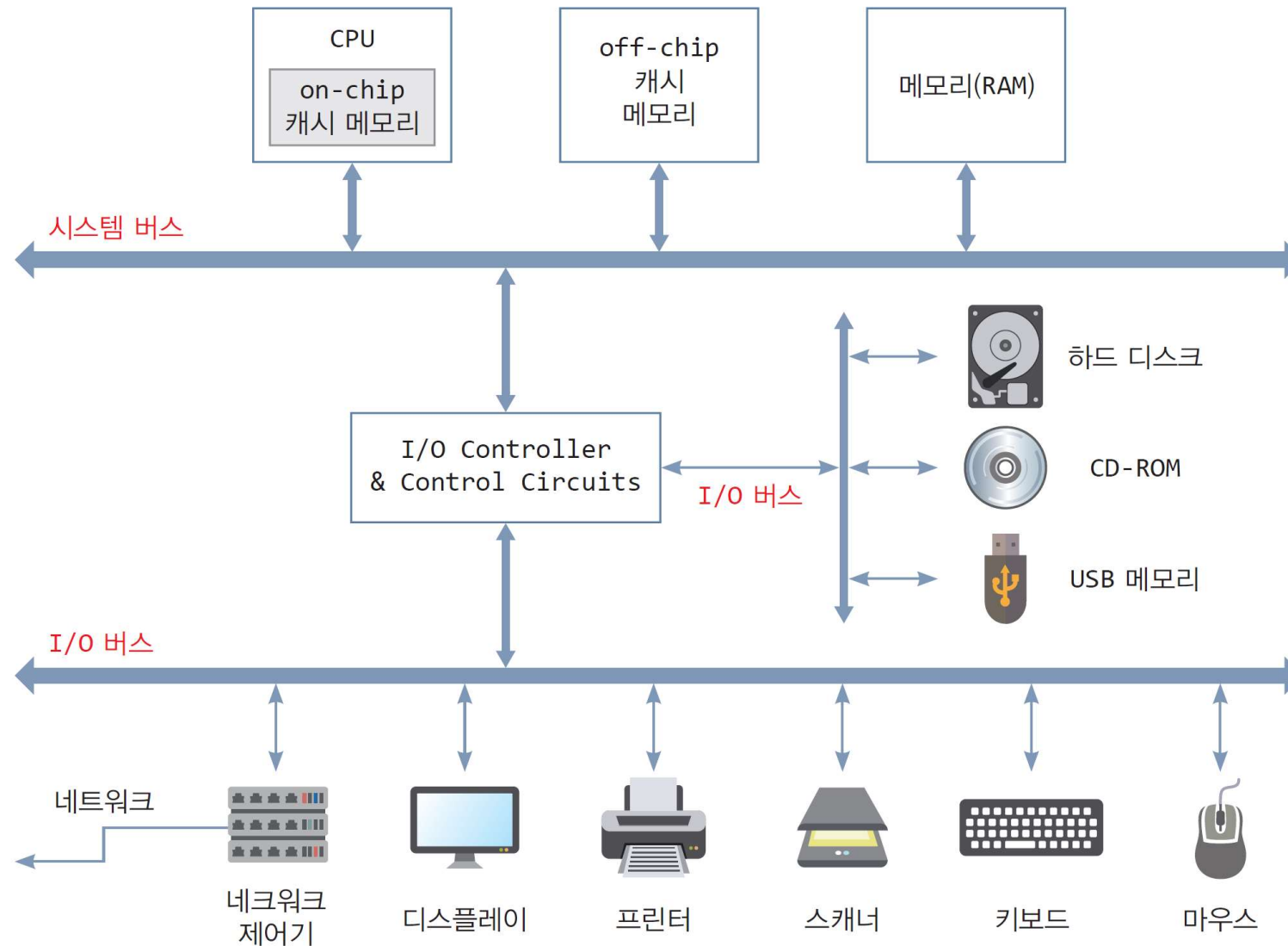
컴퓨터 시스템의 범위

5

- 컴퓨터 시스템의 계층
 - ▣ 응용프로그램 층
 - ▣ 운영체제 층
 - ▣ 컴퓨터 하드웨어 층
- 컴퓨터 시스템 계층 구조의 특징
 - ▣ 사용자는 응용프로그램과 GUI/도구프로그램(툴 /유틸리티)을 통해 컴퓨터 활용
 - ▣ 하드웨어는 모두 운영체제의 배타적 독점적 지배 받음
 - ▣ 사용자나 응용프로그램의 하드웨어에 대한 직접 접근 불허
 - 반드시 운영체제를 통해서만 접근 가능
- 계층 구조로 보는 운영체제의 기능
 - ▣ 사용자가 하드웨어에 대해 몰라도 컴퓨터를 사용할 수 있도록 함
 - ▣ 응용프로그램과 하드웨어 사이의 중계
 - 위로는 응용프로그램과 아래로는 하드웨어와의 인터페이스

컴퓨터 하드웨어 구성

6



컴퓨터 하드웨어 설명(1)

7

- CPU(Central Processing Unit)
 - ▣ 프로그램 코드(기계 명령)를 해석하여 실행하는 중앙처리장치
 - ▣ 컴퓨터의 가장 핵심 장치
 - ▣ 전원이 공급될 때 작동 시작, 메모리에 적재된 프로그램 실행
- 메모리
 - ▣ CPU에 의해 실행되는 프로그램 코드와 데이터가 적재되는 공간
 - ▣ 반도체 메모리 RAM
 - ▣ 프로그램은 실행되기 위해 반드시 메모리에 적재되어야 함
- 캐시 메모리(Cache Memory)
 - ▣ 배경
 - CPU 처리속도가 메모리 속도에 비해 빠르게 발전 -> 느린 메모리 때문에 CPU의 대기 시간이 늘게 되었음
 - CPU의 프로그램 실행 속도를 높이기 위해, CPU와 메모리 사이에 소량의 빠른 메모리(고가의 메모리)를 설치하게 되었음
 - ▣ 온칩 캐시(on-chip) – CPU 내부에 만들어진 캐시, 오늘날 대부분 온칩 캐시
 - ▣ 오프칩 캐시(off-chip) – CPU 외부에 설치되는 캐시
 - ▣ 캐시 메모리가 있는 경우 CPU는 캐시 메모리에서만 프로그램 실행
 - 실행하고자하는 프로그램과 데이터는 메모리에 먼저 적재되고 다시 캐시로 옮겨져야 함
 - 캐시는 용량이 작기 때문에 현재 실행할 코드와 데이터의 극히 일부분만 저장

컴퓨터 하드웨어 설명(2)

8

- 장치들
 - ▣ 키보드, 프린터, 스캐너 등
- 버스(bus)
 - ▣ 하드웨어들이 데이터를 주고받기 위해 0과 1의 디지털 신호가 지나가는 여러 가닥의 선을 다발로 묶어 부르는 용어
 - ▣ 비유
 - 컴퓨터의 버스 : 도로
 - 컴퓨터의 데이터 : 도로에 다니는 자동차
 - ▣ 버스의 종류(버스에 지나다니는 정보에 따라)
 - 주소 버스(address bus) - 주소 신호가 지나다니는 버스(도로)
 - 데이터 버스(data bus) - 데이터 신호가 지나다니는 버스(도로)
 - 제어 버스(control bus) - 제어 신호가 지나다니는 버스(도로)
 - ▣ 주소
 - 메모리, 입출력 장치나 저장 장치 내에 있는 저장소(레지스터들)에 대한 번지
 - 0번지에서 시작하는 양수
 - 주소 버스는 주소 값이 전달되는 여러 선의 다발
 - **CPU**는 메모리나 입출력 장치에 값을 쓰거나 읽을 때 반드시 **주소를 발생시킴**

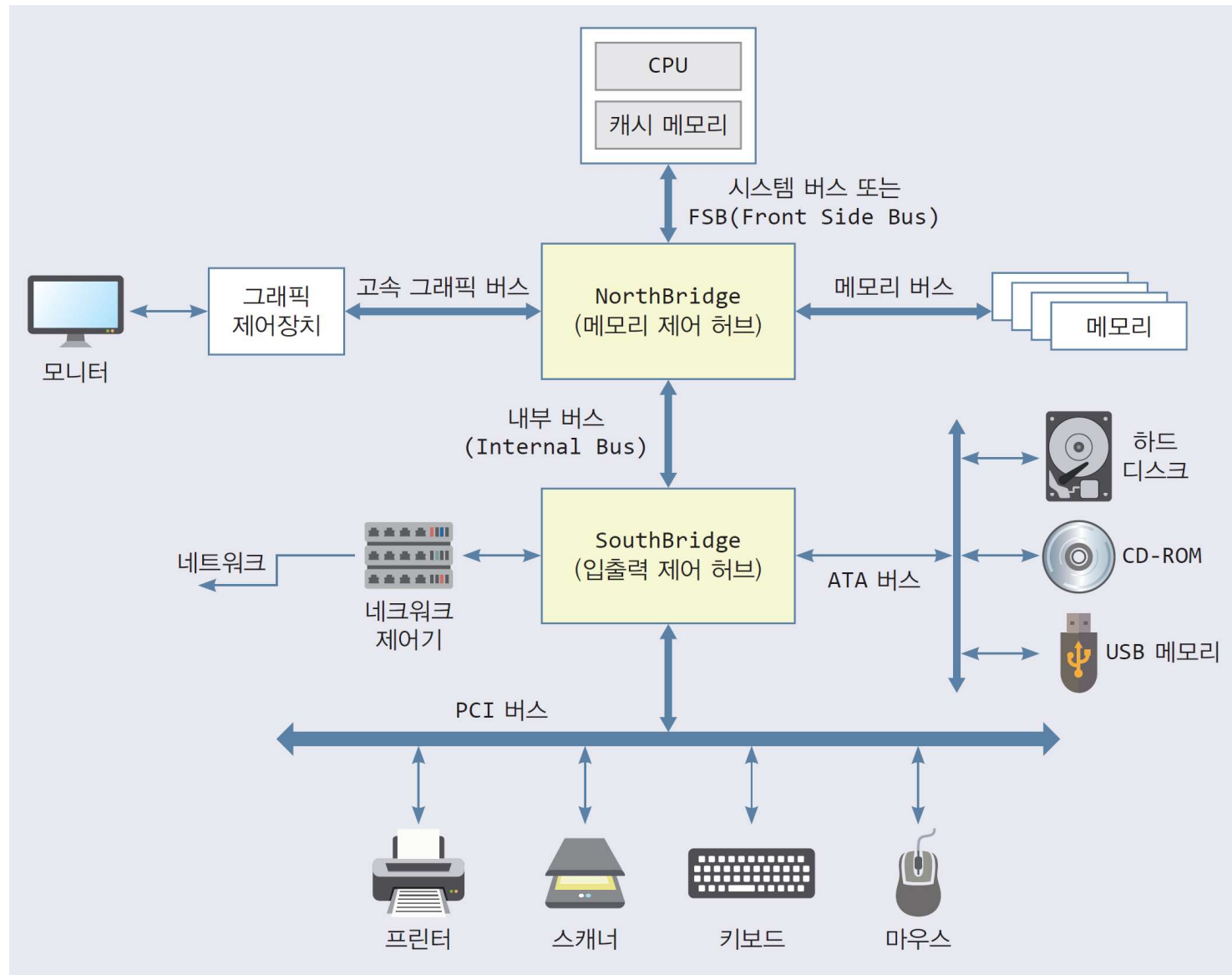
컴퓨터 하드웨어 설명(3)

9

- 목적에 따라 버스 구분
 - ▣ 시스템 버스(system bus)
 - CPU, 캐시 메모리, 메모리 등 빠른 하드웨어들 사이에 데이터 전송
 - **고속도로**에 비유
 - ▣ 입출력 버스(I/O bus)
 - 상대적으로 느린 입출력 장치들로부터 입출력 데이터 전송
 - **일반 도로**에 비유
- I/O controllers & control circuit
 - ▣ 입출력 장치들을 제어하기 위한 여러 하드웨어
 - 입출력 장치에게 명령 하달
 - 메모리와 입출력 장치 사이에 혹은 CPU와 입출력 장치 사이에 데이터 전달 중계
 - DMAC(Direct Memory Access Controller), 인터럽트 제어장치 (Interrupt Controller, INTC) 등 포함

현대 PC의 구조

10



CPU와 메모리의 관계

11

□ CPU

- ▣ 능동적 소자. 메모리 액세스 시 주소 발생

□ 32비트 CPU, 32비트 운영체제, 32비트 컴퓨터란?

1) CPU에 **32개의 주소선** 있음

- CPU의 액세스 범위 : 2^{32} 개의 서로 다른 주소($0 \sim 2^{32}-1$ 번지)
- CPU가 최대 액세스할 수 있는 메모리의 크기 : 4GB
- 한 번지의 공간이 1바이트이므로, **2^{32} 개의 주소 = 2^{32} 바이트 = 4GB**

- $2^{10} = 1\text{KB}$
- $2^{20} = 2^{10} \times 2^{10} = 1\text{MB}$
- $2^{30} = 2^{20} \times 2^{10} = 1\text{GB}$
- $2^{32} = 2^2 \times 2^{30} = 4 \times 1\text{GB} = 4\text{GB}$

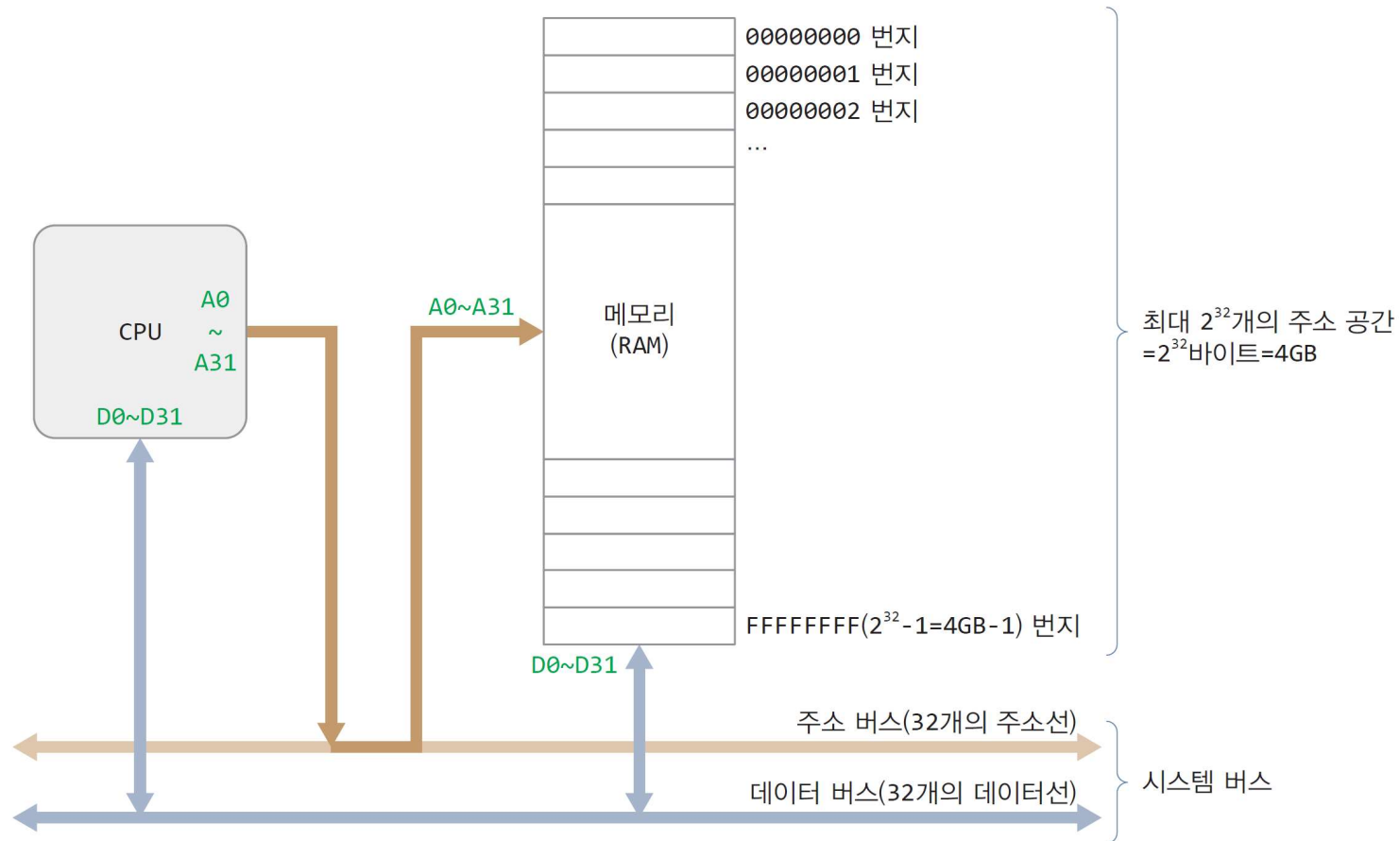
- 32비트 CPU를 가진 컴퓨터에 4GB이상 메모리를 설치할 경우
 - ▣ 4GB를 넘어선 영역은 사용할 수 없음

2) CPU에 입출력되는 **32개의 데이터 선** 있음

- 한 번에 32비트 읽고 쓰기 가능(32비트는 4개의 주소)

32비트 CPU의 주소선, 데이터선과 메모리

12



- ※ CPU에서 $A0 \sim A31$ 은 32개의 주소선을, $D0 \sim D31$ 은 32개의 데이터선을 나타낸다. 이들은 시스템 버스와 연결되어 다른 하드웨어 장치들에게 주소를 보내거나 데이터를 주고받는데 사용된다.
- ※ 주소에 초점을 맞추기 위해 32비트 컴퓨터를 32개의 주소선을 가진 것으로 설명하였지만, 32비트 CPU는 32개 데이터선을 통해 32비트를 한 번에 메모리에서 읽고 쓰고 한 번에 32비트 더하기를 한다.

CPU 기계 명령(instruction)

13

□ CPU 명령

▣ CPU가 해석하고 실행할 수 있는 기계 명령(machine instruction)

- C 언어나 자바의 프로그램 소스 코드와 다름
- CPU를 설계하는 기업이 명령어들, 명령어 개수, 형태 등을 결정
- CPU의 명령 개수는 수십개~수백개
- CPU마다 명령 이름, 기계어 코드, 크기, 개수 등이 다름
- CPU 사이에 명령들의 호환성 없음
 - C 프로그램을 어떤 CPU를 대상으로 컴파일하였는지에 따라 기계어가 달라지므로, 컴파일된 코드는 CPU 사이에 호환성 없음

▣ CPU 명령 사례

어셈블리어 명령	기계 명령	
mov ecx, 51	; b9 33 00 00 00	ecx 레지스터에 51 저장
add ax, 8	; 83 c4 08	ax 레지스터에 8 더하기
push ebp	; 55	ebp 레지스터의 값을 스택에 저장
call _printf	; e8 00 00 00 00	_printf 함수 호출
ret 0	; c3	이 함수를 호출한 곳으로 리턴(0 값과 함께)

```
#include <stdio.h>
```

```
int main() {  
    int n = 10;  
    printf("hello");  
}
```

hello.c를 비주얼
스튜디오에서
컴파일한 코드

_main PROC

; COMDAT

; 3 : int main() {

컴파일된
기계 명령들

어셈블리어
명령들

논리 주소

CPU가 처리
하는 명령

00000 55
00001 8b ec
00003 81 ec cc 00 00 00
00009 53
0000a 56
0000b 57
0000c 8d bd 34 ff ff ff
00012 b9 33 00 00 00
00017 b8 cc cc cc cc
0001c f3 ab rep

push ebp
mov ebp, esp
sub esp, 204 ; 000000ccH
push ebx
push esi
push edi
lea edi, DWORD PTR [ebp-204]
mov ecx, 51 ; 00000033H
mov eax, -858993460 ; ccccccccH
stosd

; 4 : int n = 10;

0001e c7 45 f8 0a 00 00 00 mov DWORD PTR _n\$[ebp], 10 ; 0000000aH

; 5 : printf("hello");

00025 68 00 00 00 00 push OFFSET ??_C@_05CJBACGMB@hello?\$AA@
0002a e8 00 00 00 00 call _printf
0002f 83 c4 04 add esp, 4

; 6 : }

00032 33 c0 xor eax, eax
00034 5f pop edi
00035 5e pop esi
00036 5b pop ebx
00037 81 c4 cc 00 00 00 add esp, 204 ; 000000ccH
0003d 3b ec cmp ebp, esp
0003f e8 00 00 00 00 call __RTC_CheckEsp
00044 8b e5 mov esp, ebp
00046 5d pop ebp
00047 c3 ret 0

_main ENDP

CPU의 일생 – 명령 처리 과정

15

- CPU 레지스터들
 - ▣ PC(Program Counter) - 다음에 실행할 명령의 메모리 주소 저장
 - ▣ IR(Instruction Register) - 현재 실행하기 위해 메모리로부터 읽어 온 명령 저장
 - ▣ SP(Stack Pointer) - 스택의 톱 메모리 주소 저장
 - ▣ 데이터 레지스터들(data registers) - 연산에 사용되거나 사용될 데이터들 저장
 - ▣ 상태 레지스터(status register) - CPU의 상태 정보나 인터럽트 금지 등의 제어 정보 저장
 - ▣ 기타 여러 레지스터 - 페이지 테이블이 저장된 메모리 주소를 가리키는 레지스터 등
- CPU 명령 사이클(instruction cycle) – CPU의 일생
 - ▣ CPU가 하나의 명령을 실행하는 과정. CPU는 전원이 켜진 후 단순하게 **명령 사이클 반복**

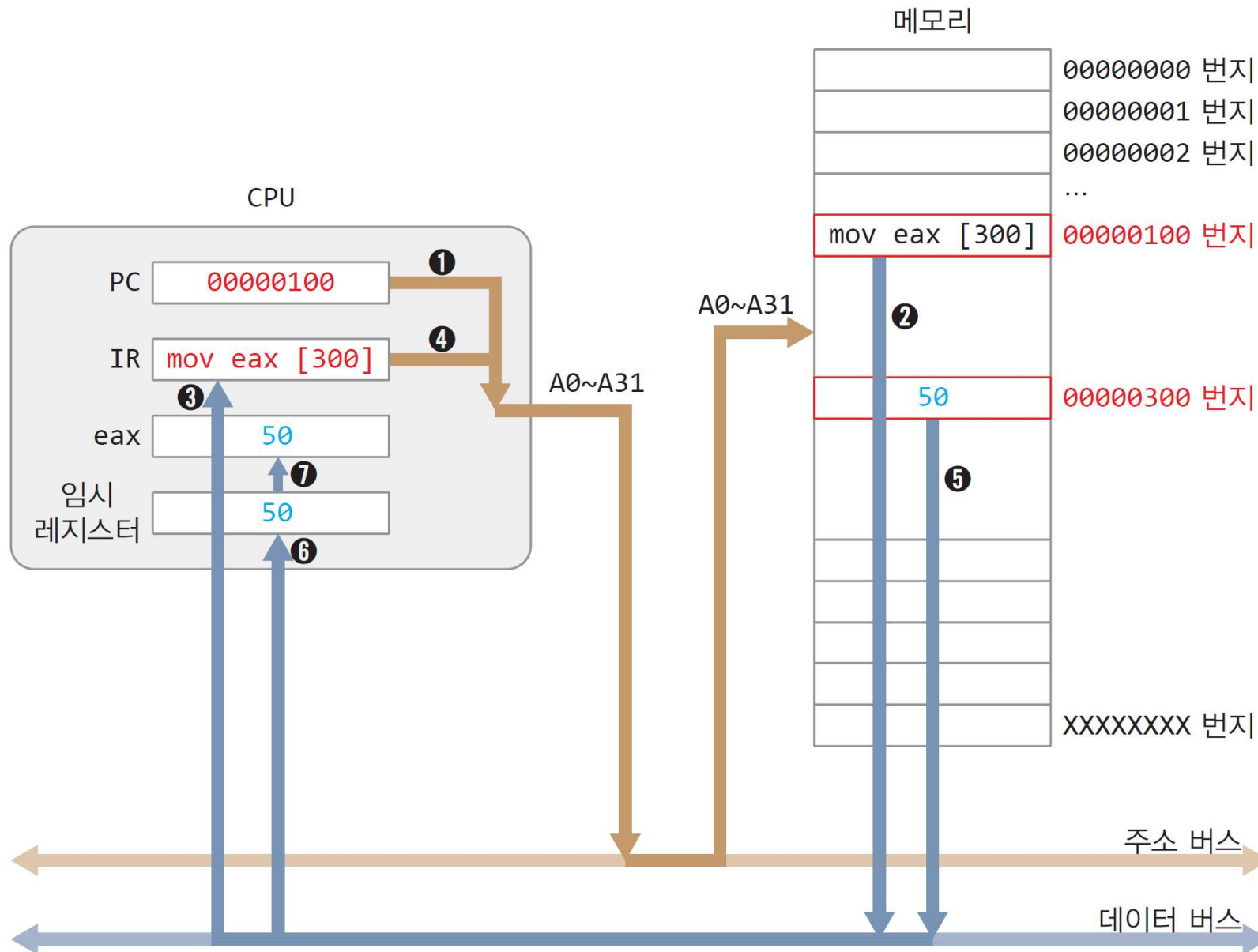
- 명령 사이클 사례 : 메모리 100번지에 저장된 다음 명령을 실행하는 사례

```
mov eax, [300]           ; 메모리 300 번지의 값을 읽어 eax 레지스터에 저장
```

- 1) CPU는, PC 레지스터가 가리키는 주소(100)를 주소 버스에 싣는다.
- 2) 메모리는, 100번지에 저장된 데이터(mov eax, [300])를 데이터 버스에 싣는다.
- 3) CPU는, 데이터 버스에 담긴 바이너리 값들을 IR 레지스터에 저장. PC를 다음 번지로 수정한다.
- 4) CPU는, 연산에 필요한 데이터를 읽기 위해 데이터의 주소(300)를 주소 버스에 싣는다.
- 5) 메모리는, 300번지에 저장된 50을 데이터 버스에 싣는다.
- 6) CPU는, 데이터 버스로부터 50을 임시 데이터 레지스터에 저장된다.
- 7) 이제 CPU는, 명령을 해석하고 명령을 실행한다. 명령 실행 결과, 50이 eax 레지스터에 저장된다.

CPU의 명령 사이클 - CPU가 명령을 실행하는 과정 - CPU와 메모리 사이의 관계 이해

16



스택(Stack)은 어디에 있는가?

17

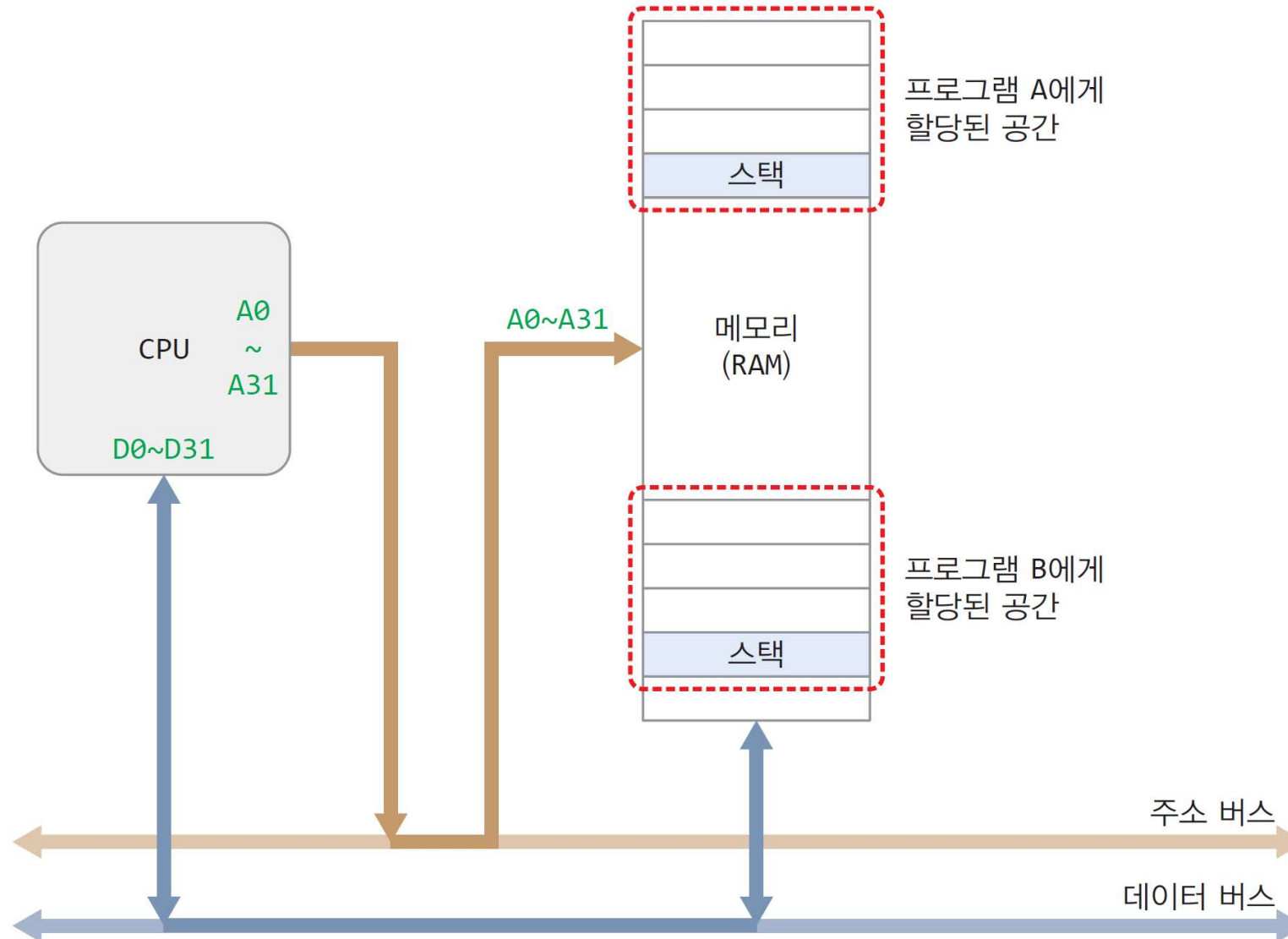
- 프로그램이 실행되기 위해 운영체제에 의해 할당되는 4공간
 - ▣ 코드(code) 공간 - 프로그램 코드 적재
 - ▣ 데이터(data) 공간 - 전역 변수들이 적재되는 공간
 - ▣ 힙(heap) 공간 - 프로그램에서 동적 할당받는 공간
 - ▣ 스택(stack) 공간 - 함수가 호출될 때 매개변수, 지역변수 등 저장

- 스택
 - ▣ 메모리의 일부를 스택으로 사용하도록 할당된 공간
 - 스택이라는 별도의 하드웨어 메모리가 있는 것 아님
 - ▣ 운영체제는 각 프로그램에게 스택 공간 할당
 - ▣ CPU의 SP 레지스터는 현재 CPU가 실행중인 프로그램의 스택 꼭대기 주소를 가리킴
 - ▣ 스택에 저장되는 내용
 - 함수의 지역 변수들
 - 함수가 호출될 때 전달받은 매개변수 값들
 - 함수가 실행된 후 돌아갈 주소
 - 함수가 의도적으로 저장해 두기 위한 값

스택과 메모리

18

스택은 운영체제에 의해 프로그램마다 할당된 메모리의 일부 영역



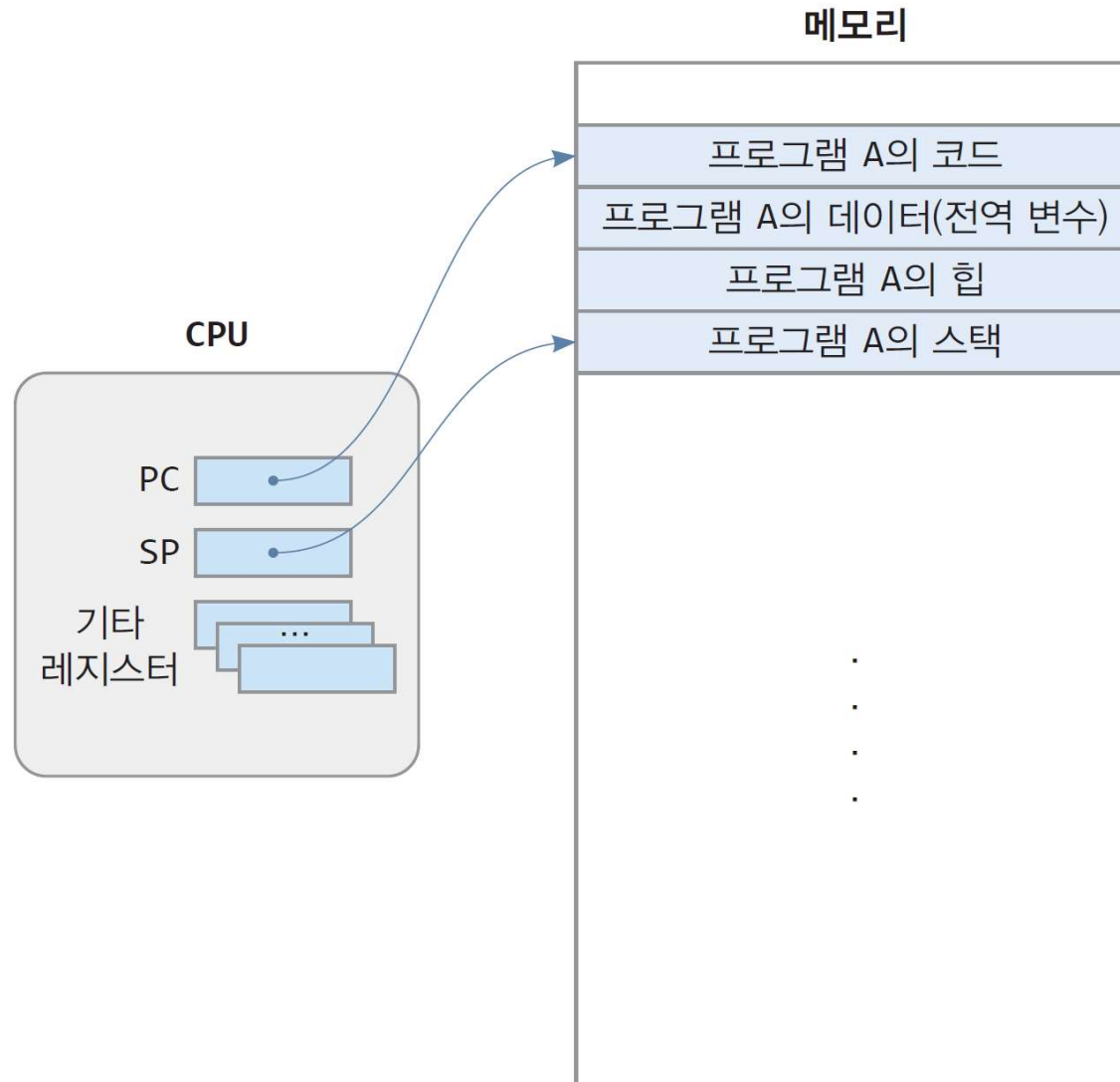
컨텍스트(Context)

19

- 컨텍스트(문맥이라고도 표현)
 - ▣ 한 프로그램이 실행 중인 일체의 상황 혹은 상황 정보
 - 메모리
 - 프로그램 코드와 데이터, 스택, 동적할당 받아 저장한 값
 - CPU 레지스터들의 값
 - PC에는 코드의 주소
 - SP에는 스택의 주소
 - 다른 레지스터는 이전의 실행 결과나 현재 실행에 사용되는 데이터 등
 - ▣ 축소 정의
 - 현재 CPU에 들어 있는 레지스터의 값들
 - 메모리에 저장된 상황 정보는 그대로 있으니까
- 컨텍스트 스위칭
 - ▣ 발생
 - CPU가 현재 프로그램의 실행을 중지하고 다른 프로그램을 실행할 때
 - ▣ 과정
 - 현재 실행 중인 프로그램의 컨텍스트(CPU레지스터들의 값)를 메모리에 저장
 - 새로 실행시킬 프로그램의 저장된 컨텍스트(CPU레지스터들의 값)를 CPU에 복귀

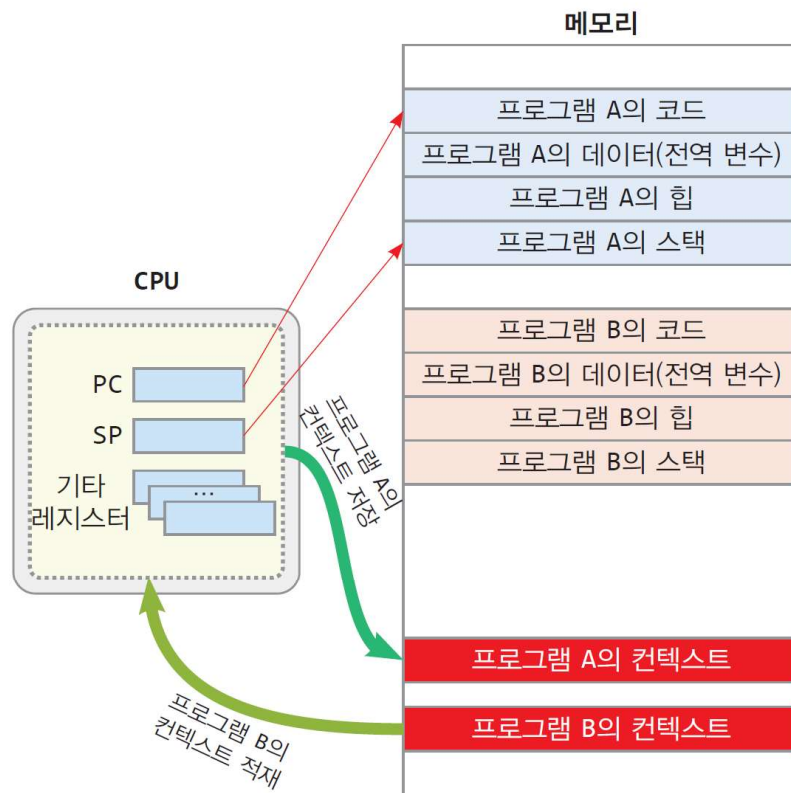
프로그램 A의 컨텍스트 사례

20

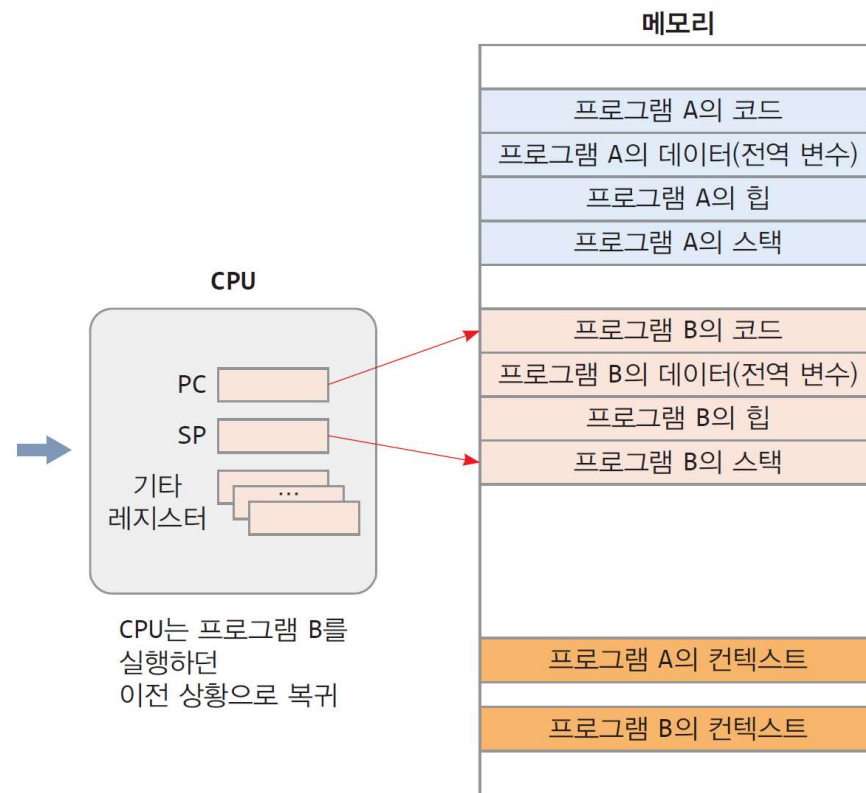


프로그램 A에서 프로그램 B로 컨텍스트 스위칭

21



(a) 프로그램 A에서 프로그램 B로 컨텍스트 스위칭

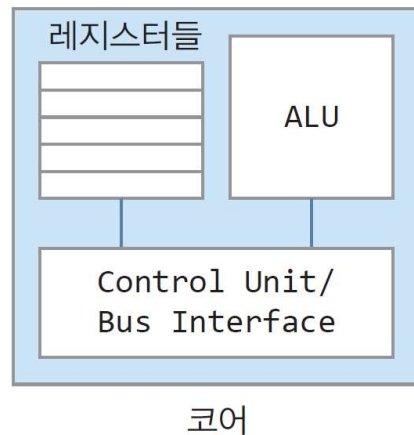


(b) 컨텍스트 스위칭 후 프로그램 B 실행 재개

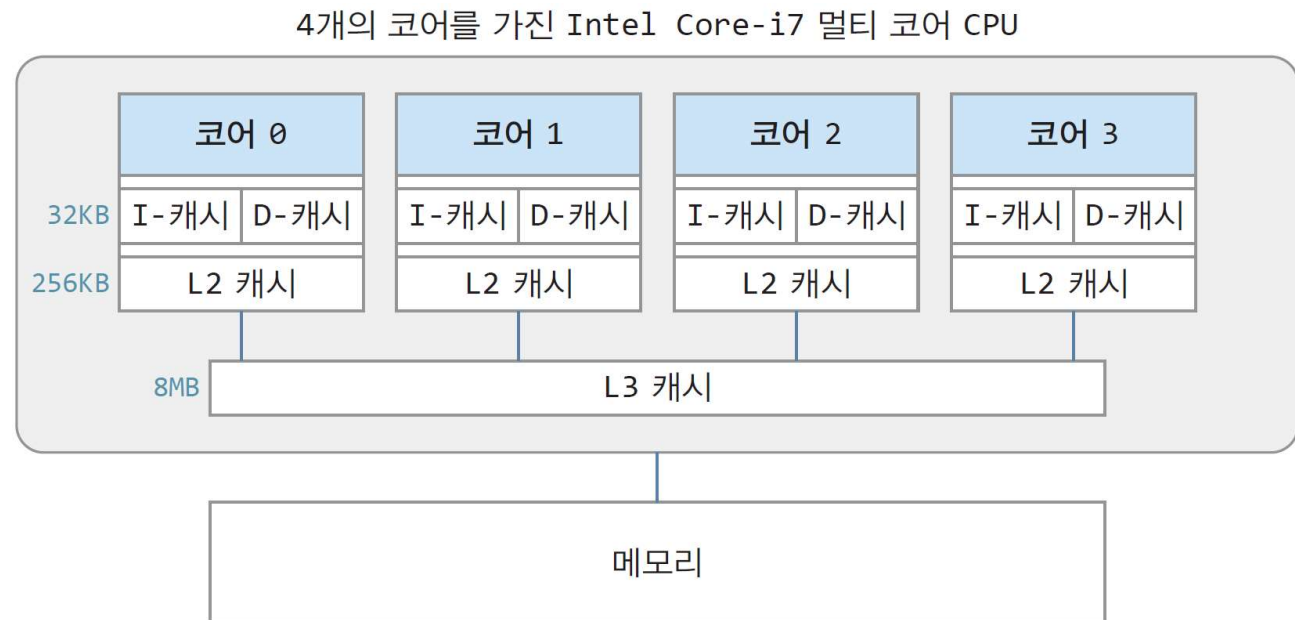
멀티 코어 CPU(Multi-core CPU)

22

- 2001년 IBM에 의해 PowerPC라는 멀티코어 CPU 개발
 - ▣ CPU 내부에 2개의 프로세서(processor) 포함
 - ▣ 2개의 프로그램을 동시에 실행
 - ▣ 코어는 완벽한 처리기(과거 개념의 CPU)



(a) 코어의 내부 구조



(b) 4개의 코어를 가진 Intel Core-i7 멀티 코어 CPU 사례

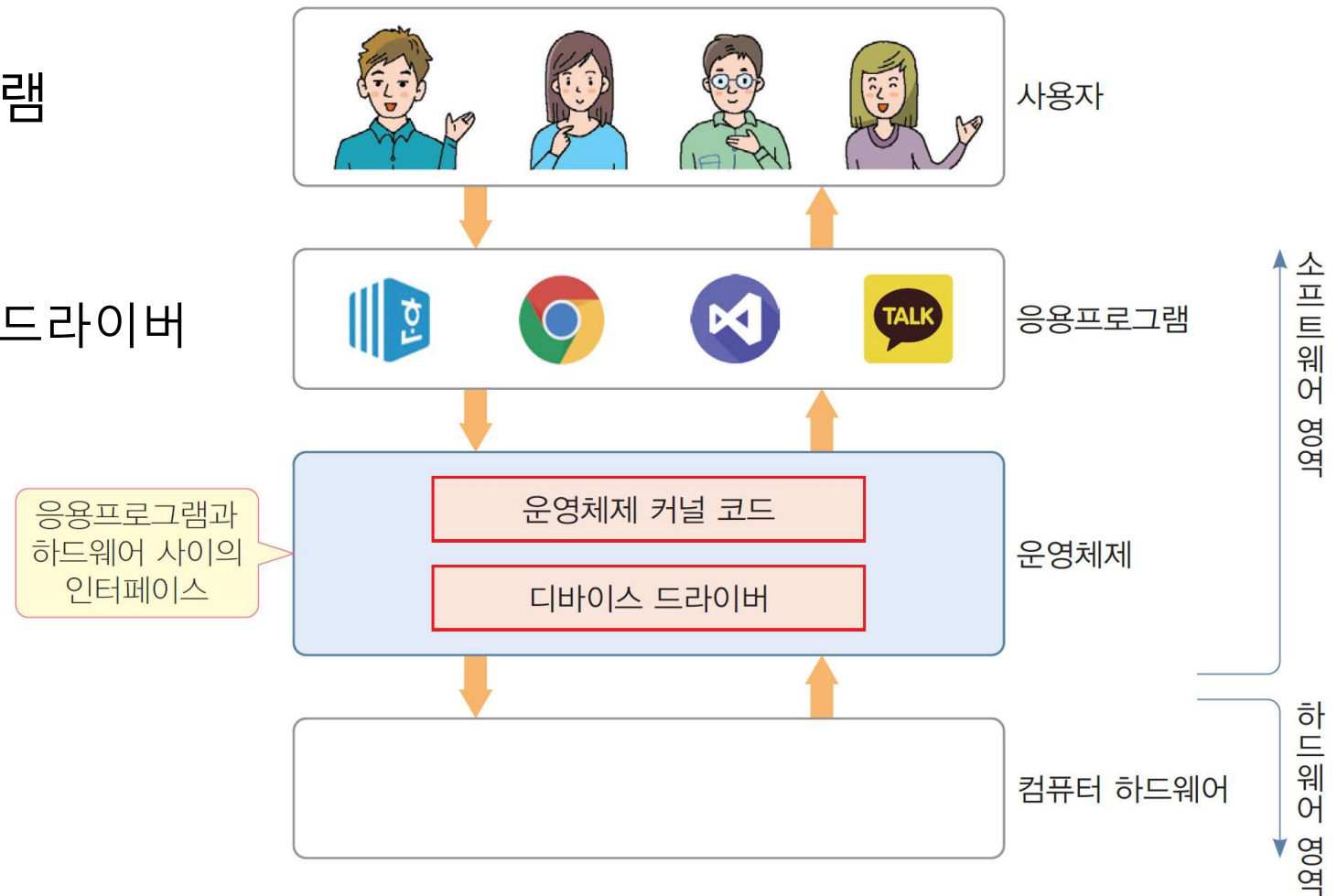
2. 컴퓨터 시스템의 계층 구조와 운영체제 인터페이스

컴퓨터 시스템 계층 구조

24

□ 컴퓨터 시스템 계층

- ▣ 사용자
- ▣ 응용프로그램
- ▣ 운영체제
 - 커널 코드
 - 디바이스 드라이버
- ▣ 하드웨어



컴퓨터 시스템이 계층 구조로 설계된 이유

25

- 계층간 독립성 확보(칸막이가 있다고 생각)를 위해
 - ▣ 사용자
 - 운영체제나 하드웨어에 대해 몰라도 응용프로그램으로 컴퓨터 활용
 - ▣ 응용프로그램
 - 컴퓨터 하드웨어의 타입이나 구조, 제어 방법을 몰라도 개발 가능
 - 예)
 - CPU 크기, 메모리 크기가 얼마인지 모르고 프로그램 작성
 - 저장 장치가 하드디스크인지 SSD인지, 저장 장치의 크기는 얼마인지, 디스크 헤드는 몇개 있는지 몰라도 파일 입출력 프로그램 작성
 - 운영체제에게 요청하여 해결
 - 컴퓨터 하드웨어가 바뀌어도 응용프로그램을 다시 작성할 필요 없음
 - ▣ 운영체제
 - 운영체제는 장치 관련된 모든 작업을 디바이스 드라이버에게 요청
 - 응용프로그램과 하드웨어 사이의 인터페이스

왜 운영체제가 필요한가?

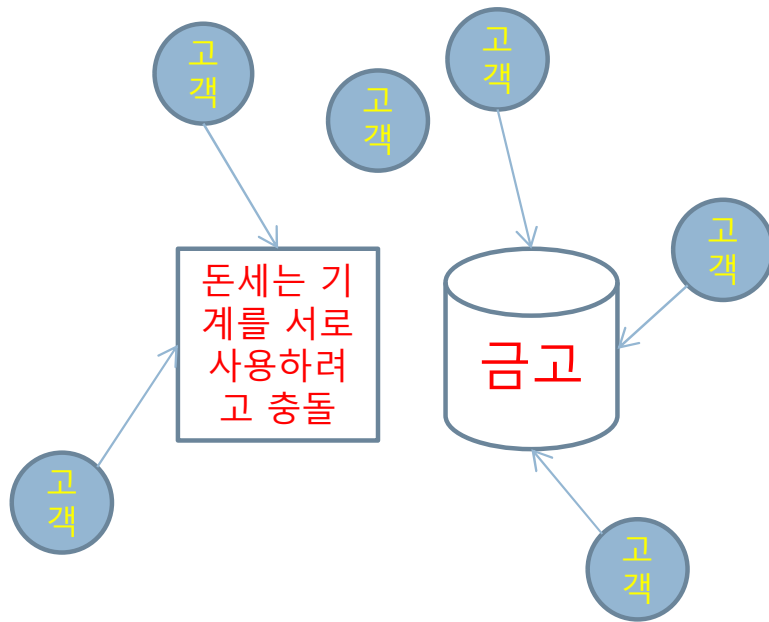
26

- ▣ 운영체제가 없다면
 - 응용프로그램이나 사용자가 직접 하드웨어를 제어해야 함
 - 하드웨어에 대한 지식이 필요하며, 충돌, 관리, 보안의 문제 발생
- ▣ 실례
 - 2 명이 동시에 **프로그램을 실행시키고자** 할 때, 이미 어떤 프로그램이 실행되고 있는 상황에서, 다른 사람이 프로그램을 실행시키고자 하면, 2개의 응용프로그램에 대한 스케줄링은 누가 할 것인가? **프로세스 관리**
 - 응용프로그램이 **메모리가 필요할** 때, 사용 중이지 않는 메모리와 사용 중인 메모리로 나누어 관리하고, 응용 프로그램이 종료하면 사용한 메모리 회수하는 등 메모리 관리는 누가 할 것인가? **메모리관리**
 - 2명의 사용자가 동시에 프린터에 **프린팅을** 시킬 때 등, 장치 사용에 대한 충돌이 생기지 않게 하려면 누가 할 것인가? **장치 관리**
 - 데이터를 **파일에 기록하려는데**, 하드 디스크의 어느 위치에 기록해야 할지 결정해야 하고, 내가 기록한 디스크 위치에 다른 응용프로그램이 덮어쓰지 못하게 하려면 누가 할 것인가? **파일 시스템 관리**
 - 키보드에서 **키가 입력되면** 실행 중인 여러 응용프로그램 중 어떤 응용프로그램에게 키를 전달할지 누가 결정할 것인가? **입출력 관리**
- ▣ 운영체제의 필요성
 - 자원에 대한 충돌 해결, 성능 최적화, 사용자의 시스템 사용 효율화

직원이 없는 은행과 직원 있는 은행 비교

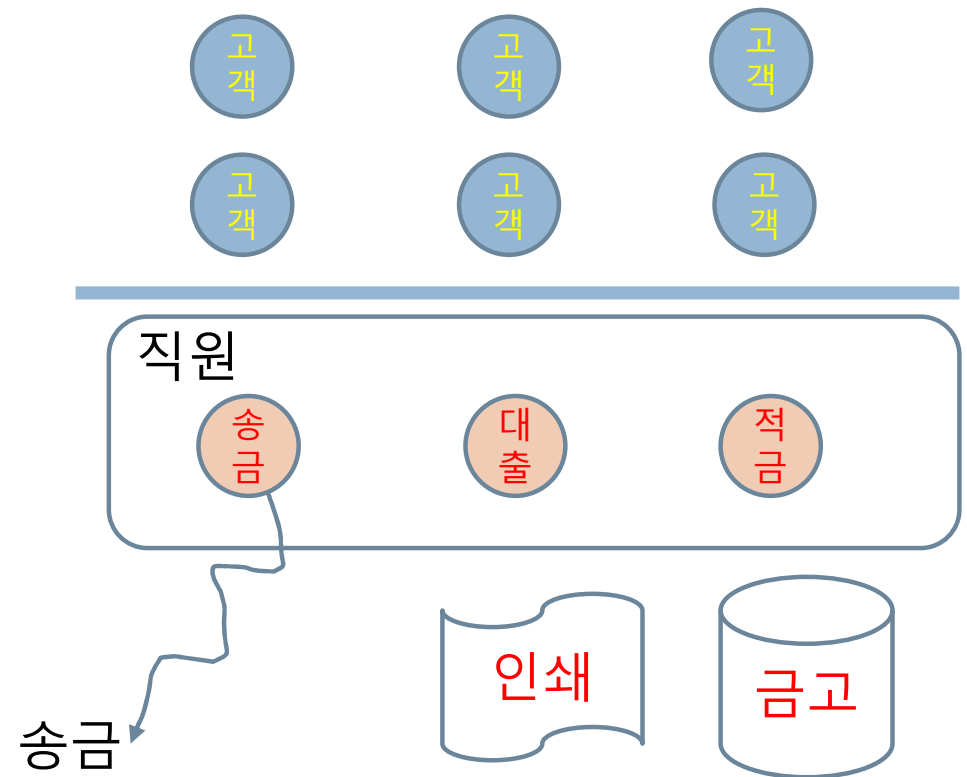
27

은행직원이 없는 경우



비효율성, 충돌로 인한 오류, 지연,
보안 취약, 무질서 !!!

은행직원이 있는 경우



- 고객 : 응용프로그램
- 직원 : 운영체제의 여러 기능들(커널 함수들)

효율적, 충돌 해소, 보안 강화, 질서

- 사용자 모드 : 고객이 활동하는 영역
- 커널 모드 : 직원이 활동하는 영역

운영체제와 응용프로그램 사이의 관계

28

□ 응용프로그램

- ▣ 워드, 웹브라우저 등, 사용자가 컴퓨터를 활용하도록 작성된 다양한 프로그램들

□ 응용프로그램에 대한 운영체제의 역할

- ▣ 응용프로그램이 직접 하드웨어를 다루지 못하도록 차단
 - 운영체제가 하드웨어를 완벽히 독점 장악
 - 이유 : 응용프로그램들 사이의 하드웨어 사용 충돌을 막기 위함
- ▣ 응용프로그램이 하드웨어를 사용하고자 할 때
 - 반드시 운영체제에게 요청 -> 운영체제가 대신하여 하드웨어 조작
 - 유일한 요청 방법 - **시스템 호출(system call)**
- ▣ 응용프로그램과 하드웨어 사이의 인터페이스
- ▣ 응용프로그램들의 실행 순서 제어
- ▣ 응용프로그램들 사이의 통신 중계

운영체제와 사용자의 관계

29

- 사용자는 응용프로그램을 통해 컴퓨터 활용
 - ▣ 탐색기, 메모장 등
- 사용자에 대한 운영체제의 역할
 - ▣ 사용자가 하드웨어에 관한 지식이 없어도 컴퓨터 다루기 용이
 - ▣ 사용자가 하드웨어를 설치하거나 변경하는 것에 도움
 - ▣ 사용자에게 컴퓨터 시스템을 사용할 편리한 인터페이스 제공
 - UI, 마우스, 음성 명령 등
 - ▣ 컴퓨터의 사용을 돕는 여러 도구 응용프로그램(유틸리티) 제공
 - Windows의 탐색기와 작업 관리자
 - 리눅스의 쉘
 - ▣ 사용자 계정 관리
 - ▣ 사용자의 컴퓨터 사용 시간 계산, 과금 처리 등

운영체제와 하드웨어의 관계

30

- 하드웨어를 제어하는 것은 전적으로 운영체제의 몫
 - ▣ 응용프로그램에서 `printf("hello")`
 - 디스플레이 장치에 "hello" 를 출력하는 일은 운영체제가 함
 - ▣ 응용프로그램에서 `scanf()`
 - 키보드로부터 문자를 입력받는 일은 운영체제가 함
- 운영체제
 - ▣ 사용자/응용프로그램과 하드웨어 사이의 중계자
 - ▣ 하드웨어 제어는 전적으로 운영체제의 기능
 - 하드디스크에서 파일을 읽거나 쓰기
 - 마우스의 클릭
 - 키보드 입력 받기
 - 네트워크를 통한 데이터 전송 혹은 수신
 - 디스플레이에 텍스트나 이미지, 그래픽 등 출력

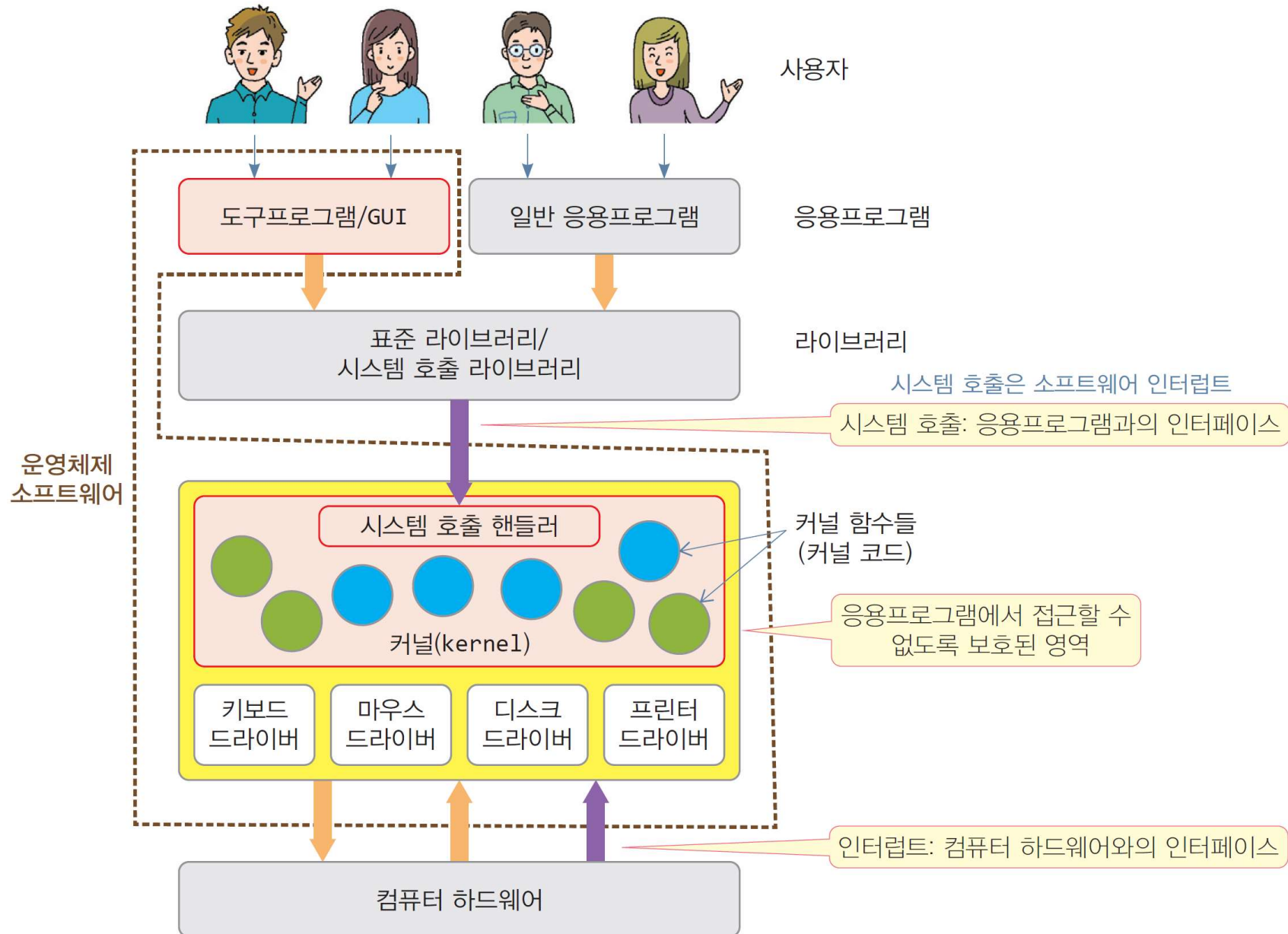
운영체제의 전체 기능

31

- 프로세스와 스레드 관리
 - ▣ 프로세스/스레드의 실행, 일시 중단, 종료, 스케줄링, 컨텍스트 스위칭, 동기화
- 메모리 관리
 - ▣ 프로세스나 스레드에게 메모리 할당, 메모리 반환, 다른 프로세스/스레드로부터의 메모리 보호
 - ▣ 메모리를 하드 디스크의 영역까지 확장하는 가상 메모리 기술
- 파일 관리 혹은 파일 시스템 관리
 - ▣ 파일 생성, 저장, 읽기, 복사, 삭제, 이동, 파일 보호
- 장치 관리
 - ▣ 키보드, 마우스, 프린터 등 입출력 장치, 하드 디스크 등 저장 장치 제어
 - ▣ 입출력
- 사용자 인터페이스
 - ▣ 라인 기반 명령 입출력 창, 마우스와 그래픽 사용 GUI 인터페이스 제공
- 네트워킹
 - ▣ 네트워크 인지, 연결, 닫기, 데이터 송수신
- 보호 및 보안
 - ▣ 바이러스나 웜, 멀웨어(malware), 해킹 등의 외부 공격이나 무단 침입으로부터 보호

운영체제의 구성 요소와 커널

32



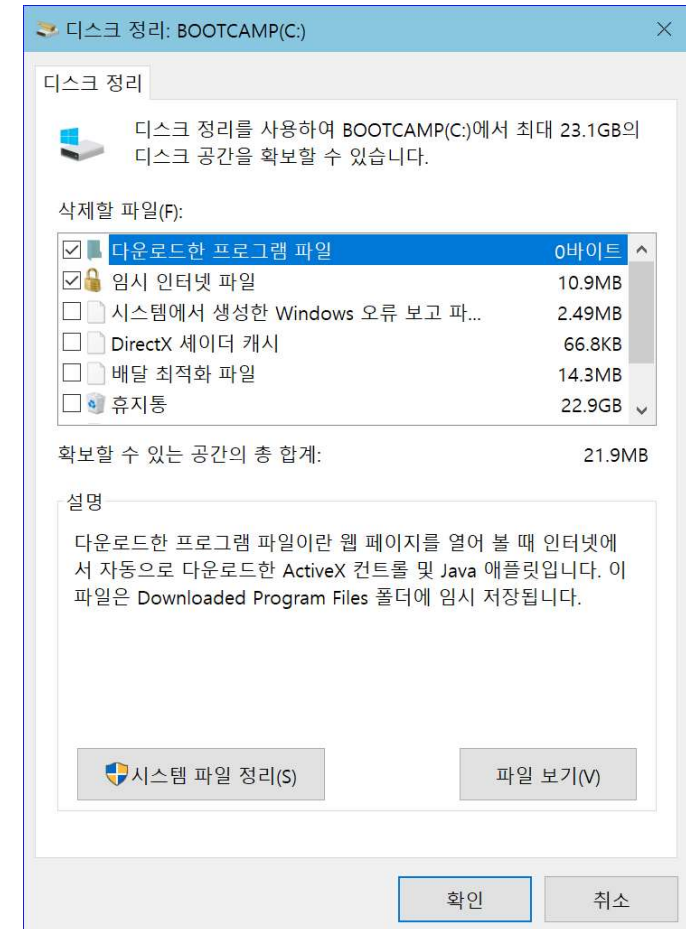
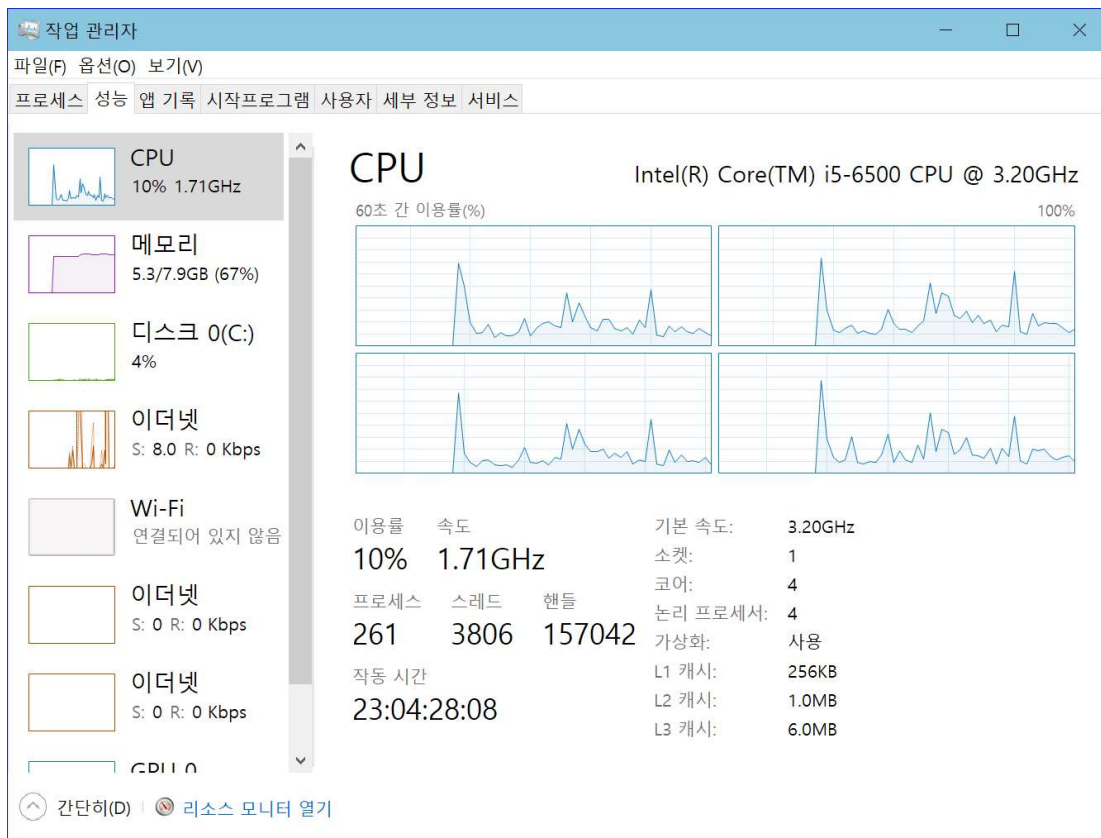
운영체제 구성

33

- 운영체제 = 커널 + 툴 + 디바이스 드라이버
 - ▣ 커널(kernel)
 - 운영체제의 핵심 부분, 좁은 의미의 운영체제
 - 부팅 후 메모리에 상주하는 코드와 데이터
 - 운영체제의 핵심 기능 모두 구현
 - CPU, Memory, MMU 등 컴퓨터 자원을 직접 제어하고 관리하는 코드와 자료 구조들
 - 커널 코드는 함수들의 집합
 - 커널 기능을 이용하려면 응용프로그램은 반드시 시스템 호출 사용
 - ▣ 도구(tool) 소프트웨어와 GUI
 - 사용자가 컴퓨터를 편리하게 사용할 수 있도록 제공하는 툴 소프트웨어 혹은 툴 응용프로그램
 - Windows 경우, 바탕화면 GUI, 탐색기, 명령창, 작업 관리자, 제어판 등
 - 리눅스 경우, 셸, 로그인 프로그램
 - ▣ 디바이스 드라이버(device driver)
 - 장치를 직접 제어하고 입출력하는 소프트웨어
 - 장치마다 전담 디바이스 드라이버 있음
 - 일반적으로 장치 제작자에 의해 작성되어 배포됨
 - 사례
 - 키보드 드라이버, 디스크 드라이버, SCSI 드라이버, 마우스 드라이버, 그래픽 드라이버, 네트워크 드라이버, usb 드라이버 등

도구 소프트웨어 사례 - 작업 관리자, 디스크 정리

34



운영체제 커널 인터페이스 : 시스템 호출과 인터럽트

35

- 커널이 제공하는 2개 인터페이스 : 시스템 호출과 인터럽트
 - 응용프로그램과 하드웨어 사이의 중계 역할
- **시스템 호출(system call)**
 - ▣ 커널과 응용프로그램 사이의 인터페이스
 - ▣ 응용프로그램에서 커널 기능을 사용할 수 있는 유일한 방법
 - ▣ 시스템 호출 라이브러리를 통해 다양한 시스템 호출 함수 제공
 - 시스템 호출 라이브러리는 운영체제 패키지에 포함됨
 - 예) 파일 읽기, 메모리 할당, 프로세스 정보 보기, 프로세스 생성 등
 - `open()`, `close()`, `read()`, `write()`, `fork()`, `exit()`, `wait()` 등의 시스템 호출 함수

표준 라이브러리(standard library)란?

- 응용프로그램을 쉽게 작성할 수 있도록 복잡한 기능이 미리 작성된 코드
 - 예) `printf()`, `abs()`, `strcmp()` 등
- 운영체제 커널의 기능과 무관한 작업
- 응용프로그램은 표준 라이브러리 함수를 함수 호출(function call) 방법으로 사용

운영체제 커널 인터페이스 - 인터럽트

36

□ 인터럽트(interrupt)

- ▣ 커널과 하드웨어 장치 사이의 인터페이스
- ▣ 장치들이 입출력 완료, 타이머 완료 등을 CPU에게 알리는 하드웨어적 방법
 - 인터럽트를 알리는 하드웨어 신호가 직접 CPU에 전달
- ▣ 인터럽트가 발생하면
 - CPU는 하는 일을 중단하고 인터럽트 서비스 루틴 실행
 - 인터럽트 서비스 루틴은 대부분 디바이스 드라이버 내에 있음
 - 예) 키를 입력하면 커널의 키보드 인터럽트 서비스 루틴 실행, 키를 읽어 커널 버퍼에 저장
 - 인터럽트 서비스 루틴은 커널 영역에 적재
 - 인터럽트 서비스 루틴의 실행을 마치면 하던 작업 계속
- ▣ 인터럽트 활용
 - 운영체제가 장치에게 지시한 입출력 작업의 완료, 예고 없는 네트워크 데이터의 도착, 키보드나 마우스의 입력, 부족한 배터리 경고 등 장치와 관련된 모든 이벤트 처리

37

3. 커널과 시스템 호출

응용프로그램의 자원 접근 문제

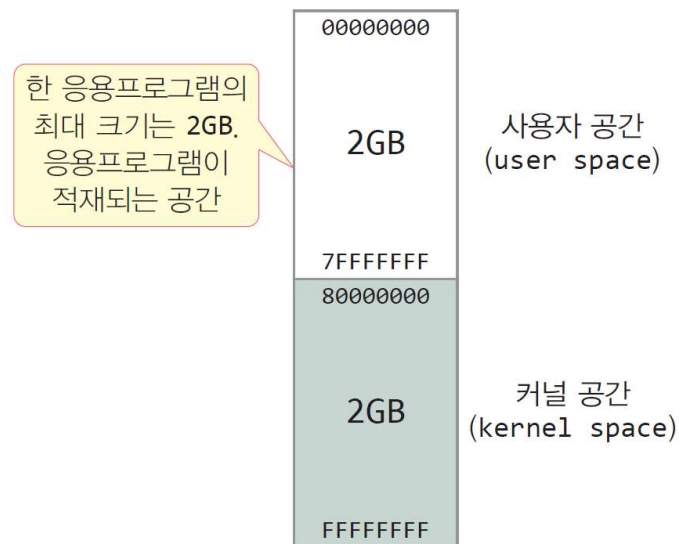
38

- 오늘날 운영체제는 다중프로그래밍 운영체제
 - ▣ 다수의 응용프로그램이 한 컴퓨터에서 동시에 실행
- 문제
 - ▣ 응용프로그램이 직접 컴퓨터 자원에 접근하면 충돌과 훼손 발생
 - 다른 응용프로그램이 적재된 메모리 훼손 가능
 - 다른 응용프로그램이 만든 파일 삭제 및 훼손 가능
 - 응용 프로그램이 커널이 적재된 영역 훼손 가능
- 해결 방안
 - ▣ 응용프로그램의 자원 접근 불허
 - 자원에 대한 모든 접근은 커널에만 부여
- 구체적인 해결 방법
 - 1) 메모리 공간을 사용자 공간과 커널 공간으로 분리
 - 응용프로그램은 사용자 공간에 적재, 커널은 커널 공간에만 적재
 - 2) CPU의 실행 모드를 사용자 모드와 커널 모드로 분리
 - 응용프로그램은 사용자 모드에서만 실행, 커널 코드는 커널 모드에서만 실행
 - 사용자 공간에서 커널 공간의 코드를 직접 접근하지 못하게 하기 위해
 - 사용자 모드에서 커널 코드를 접근하면 응용프로그램 강제 종료
 - 3) 응용프로그램이 커널 기능을 이용하고자 할 때, 시스템 호출을 이용해서만 커널 코드 이용

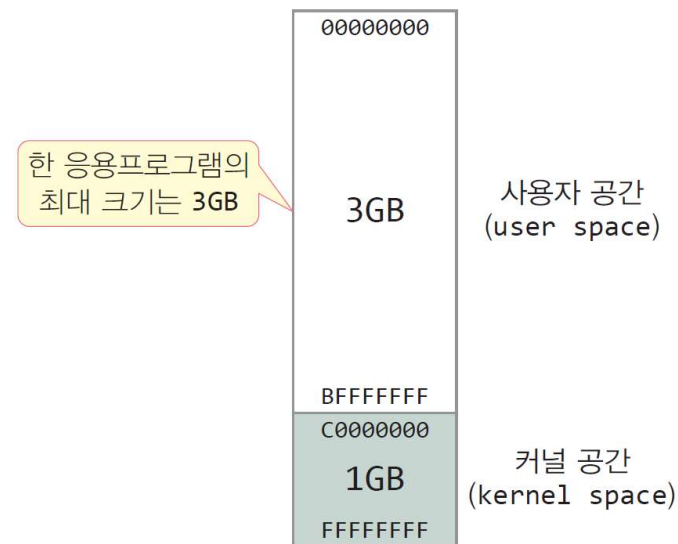
사용자 공간과 커널 공간

39

- 운영체제는 컴퓨터 메모리를 두 공간으로 분리
 - ▣ 사용자 공간(user space) : 모든 응용프로그램들이 나누어 사용하는 공간
 - 응용프로그램들이 적재되는 공간
 - ▣ 커널 공간(kernel space) : 커널만 사용할 수 있는 공간
 - 커널 코드, 커널 데이터 등 커널에 의해 배타적으로 사용되는 공간
 - 디바이스 드라이버 포함
- 분리 이유
 - ▣ 커널 코드와 데이터를 악의적인 응용프로그램이나 코딩 실수로부터 지키기 위함



(a) 32비트 Windows 운영체제의 주소 공간



(b) 32비트 리눅스 운영체제의 주소 공간

사용자 공간 크기의 의미

40

□ 사용자 공간 크기

▣ 한 응용프로그램의 최대 크기 결정

- 프로그램 코드 + 데이터(전역변수) + 힙(동적할당) + 스택 합친 크기
- 예) 32비트 Windows 운영체제에서 사용자 공간 2GB란 ->
응용프로그램을 2GB 크기 이상 개발할 수 없음

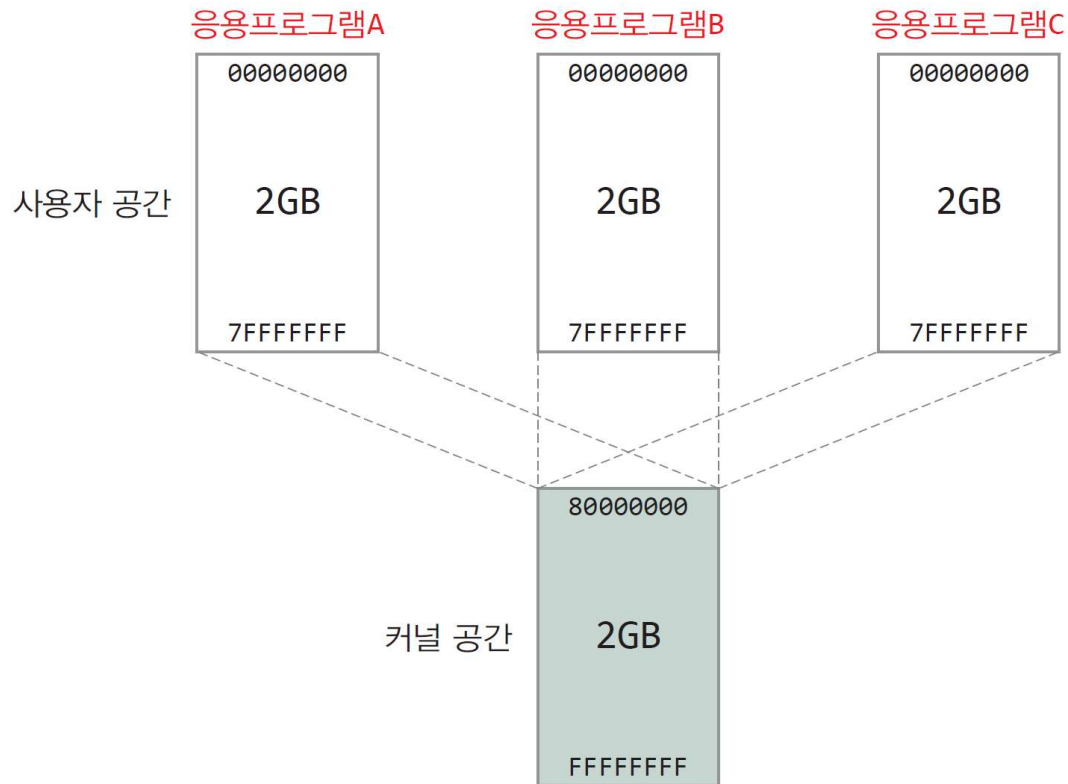
□ 사용자 공간의 주소 범위

▣ 응용프로그램은 운영체제가 설정한 사용자 공간의 주소 범위를 넘어설 수 없음

- 예) 32비트 Windows 운영체제에서,
응용프로그램은 0~7FFFFFFF 범위의 주소를 넘어 액세스하면,
바로 종료(심각한 오류)

주소 공간은 가상 주소 공간

41



- 주소 공간 = 사용자 공간 + 커널 공간
- 주소 공간은 가상 주소 공간
 - 물리 메모리의 주소 범위와 무관
- 가상 주소 공간
 - **사용자나 응용프로그램 관점에서** 보는 주소 범위
 - 사용자가 전체 메모리를 사용하고 있다고 착각
- 각 응용프로그램
 - 2GB의 사용자 주소 공간을 가짐
 - 사용자 코드와 데이터 최대 2GB
 - 0~7FFFFFFF의 2GB의 메모리 독점 사용 착각
- 커널 공간 - 2GB~4GB-1의 주소 범위
 - 80000000~FFFFFFFF 범위 주소 공간
 - 모든 응용프로그램은 **커널 공간 공유**

- 2가지 의문
 - 응용프로그램들의 **사용자 공간 충돌?**
 - 실제 물리 메모리가 **4GB보다 작은 경우** 어떻게?

2가지 의문에 대한 해결

42

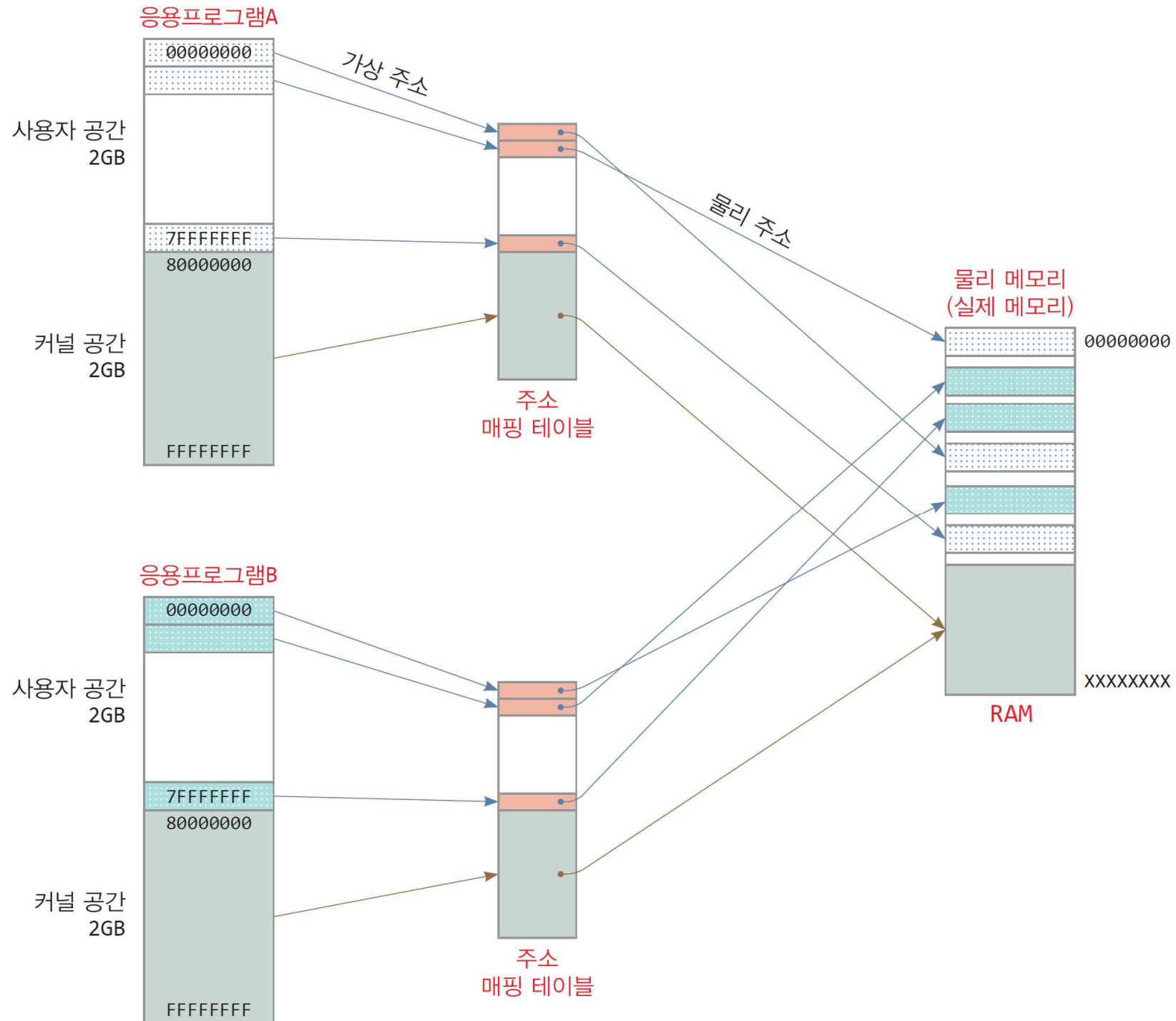
사용자 공간의 충돌 해결) 가상 주소 공간을 물리 메모리에 매핑

- ▣ 각 응용프로그램의 가상 주소 공간을 물리 주소 공간으로 매핑
 - 매핑 테이블은 운영체제가 소유하고 관리
- ▣ 물리 메모리를 여러 응용프로그램의 사용자 공간이 나누어 사용
 - 실제 각 응용프로그램은 사용자 공간의 일부만 사용
- ▣ 커널 공간 역시 물리 메모리에 매핑
 - 각 응용프로그램의 매핑 테이블에 기록
 - 커널 공간에 대한 매핑 테이블 부분을 모든 응용프로그램에서 동일

물리 메모리가 작은 경우에 대한 해결) 물리 메모리가 부족하면

운영체제는 물리 메모리를 **하드 디스크에 저장하여** 물리 메모리의 빈 영역 확보(가상 메모리 기법)

가상 주소 공간 충돌을 막기 위한 가상 주소 공간의 물리 메모리 매핑



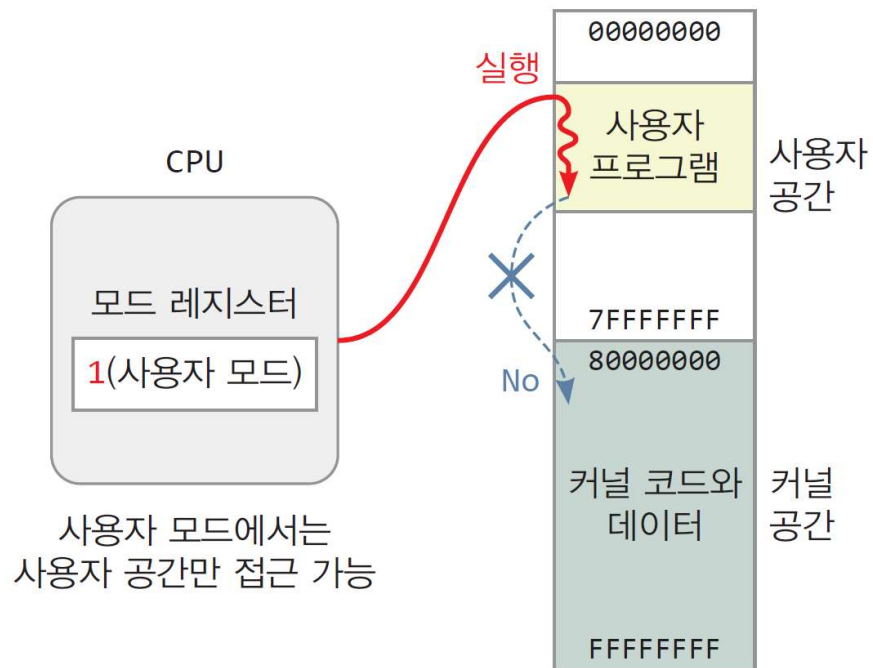
사용자 모드와 커널 모드

44

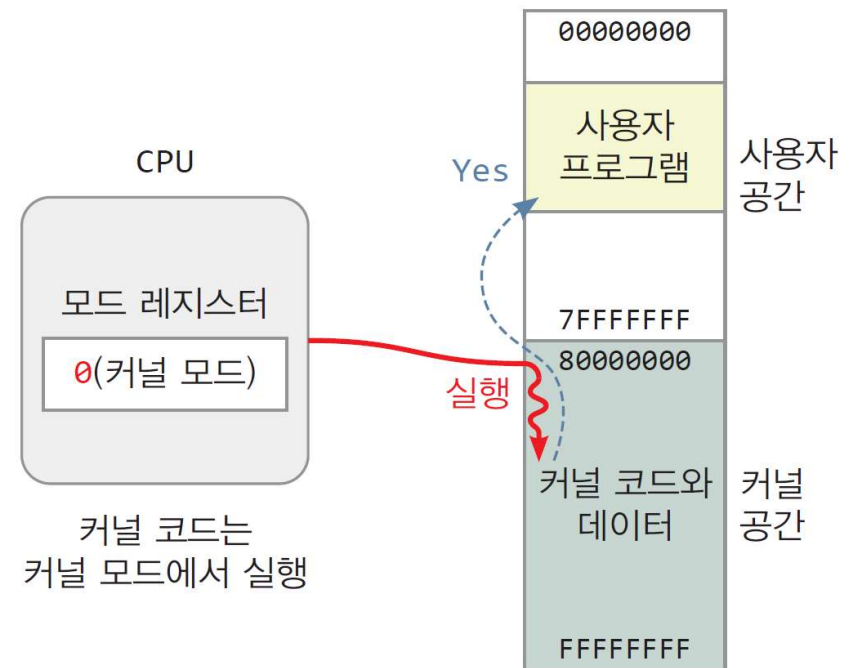
- CPU는 사용자 모드와 커널 모드 중 한 모드로 실행
 - CPU 내부에 모드 상태를 나타내는 '모드 레지스터' 있음
- ▣ 사용자 모드(user mode)
 - CPU의 모드 비트 = 1
 - CPU는 사용자 공간에 있는 코드나 데이터를 액세스하는 중
 - CPU의 커널 공간 접근 불허 -> 응용프로그램으로부터 커널영역 보호
 - 특권 명령(privileged instruction) 실행 불허
 - 특권 명령 - 입출력 장치 등 하드웨어나 시스템 중단 등 시스템 관련 처리를 위해 설계된 특별한 명령
- ▣ 커널 모드(kernel mode, supervisor mode)
 - CPU의 모드 비트 = 0
 - CPU가 커널 공간에서 실행하는 중, 혹은 사용자 코드를 실행하는 중
 - 특권 명령 사용 가능

사용자 모드와 커널 모드에서의 메모리 액세스

45



(a) 사용자 모드



(b) 커널 모드

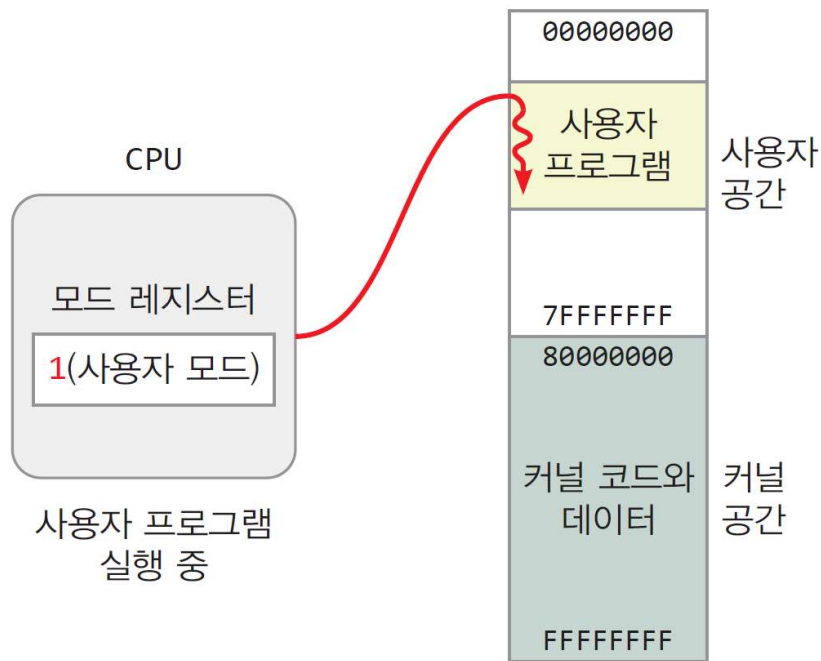
사용자 모드에서 커널 모드로 전환

46

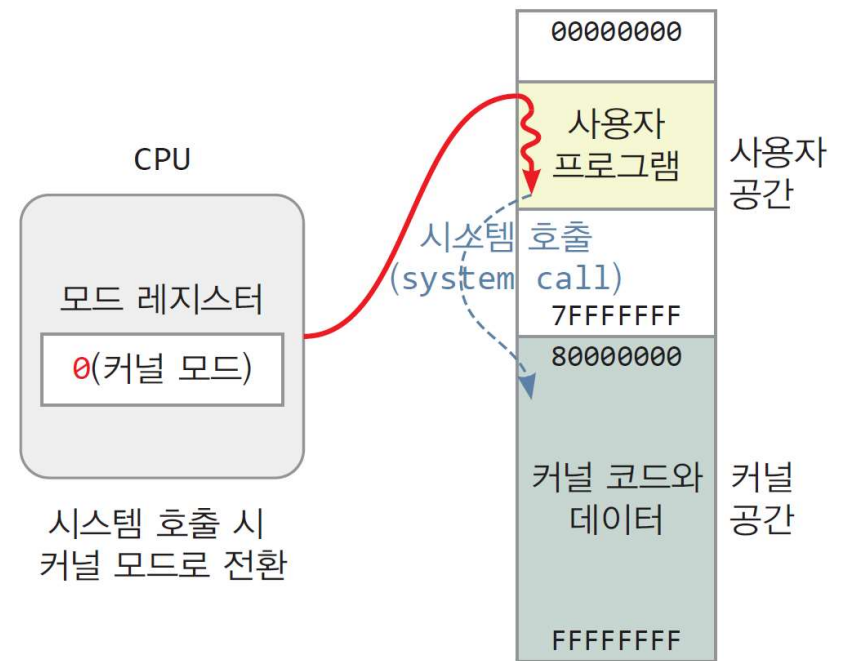
- 사용자 모드에서 커널 모드로 전환하는 경우
 - ▣ 오직 2 가지 경우 - 시스템 호출과 인터럽트 발생
- 시스템 호출
 - ▣ 시스템 호출을 실행하는 특별한 기계 명령에 의해 진행
 - 예) `int 0x80/sysenter/trap/syscall` 등 CPU마다 다름
 - ▣ 기계 명령이 CPU의 모드 비트를 커널 모드로 전환
- 인터럽트
 - ▣ CPU가 인터럽트를 수신하면 커널 모드로 자동 전환
 - 인터럽트 서비스 루틴이 커널 공간에 있기 때문
 - ▣ CPU는 인터럽트 서비스 루틴 실행
 - ▣ 인터럽트 서비스 루틴이 끝나면 CPU는 사용자 모드로 자동 전환

시스템 호출로 사용자 모드에서 커널 모드로 전환

47



(a) 사용자 모드에서 응용프로그램 코드 실행



(b) 응용프로그램이 시스템 호출을 통해 CPU의 모드를 커널 모드로 바꾸고 커널 코드 실행

사용자 모드와 커널 모드의 비교

48

	사용자 모드	커널 모드
CPU의 메모리 액세스 범위	사용자 공간에 국한. 커널 공간 액세스 불가	커널 공간을 포함한 모든 메모리 공간
CPU의 하드웨어 액세스 여부	불가(X)	모든 하드웨어 액세스 가능(0)
CPU가 처리 가능한 명령	특권 명령(privileged instruction)을 제외한 모든 CPU 명령	특권 명령을 포함한 모든 CPU 명령
오류 발생 시 처리	사용자 프로그램만 실행 종료. 시스템이 종료 되지 않으므로 안전	시스템에 심각한 오류가 발생한 것으로 시스템 종료

특권 명령

49

- 특권 명령
 - 입출력 장치로부터의 입출력(I/O), 시스템 중단, 컨텍스트 스위칭, 인터럽트 금지 등 특별한 목적으로 설계된 CPU 명령
 - 커널 모드에서만 실행
- 특권 명령 종류
 - ▣ I/O 명령
 - 하드웨어 제어 및 장치로부터의 입출력
 - 사례) `in eax, 300` ; I/O 포트 300 번지에서 값을 읽어 `eax` 레지스터에 저장
`out 301, eax` ; `eax` 레지스터에 있는 값을 I/O 포트 301 번지에 쓰기
 - ▣ Halt 명령
 - CPU의 작동을 중지시키는 명령. CPU를 유휴 상태로 만듦
 - ▣ 인터럽트 플래그를 켜고 끄는 명령
 - CPU 내에 있는 인터럽트 플래그 비트를 제어하여 CPU가 인터럽트를 허용하거나 무시하도록 지시
 - 사례) `cli/sti` 명령
 - ▣ 타이머 설정 명령
 - ▣ 컨텍스트 스위칭 명령
 - ▣ 메모리 지우기 명령
 - ▣ 장치 상태 테이블 수정 등의 명령

퀴즈 - 특권 명령에 대한 이해

50

□ Q. 다음 보기 중 어떤 명령이 특권 명령일까? 이유도 설명하라.

- 1) 사용자 모드에서 커널 모드로 전환시키는 명령
 - 특권 명령이 아니다.
 - 왜냐하면 시스템 호출을 위해 모든 응용 프로그램에게 허용되어야 하는 명령이기 때문이다.
- 2) 시계 읽기
 - 특권 명령이 아니다.
 - 모든 응용 프로그램에서 시계를 읽을 수 있어야 하기 때문이다.
- 3) 가상 메모리에서 메모리 지우기
 - 특권 명령이 아니다.
 - 프로그램이 자신의 메모리 부분을 지우는 것은 다른 프로세스의 영역을 침범하지 않기 때문이다.
- 4) 인터럽트 끄기
 - 특권 명령이다.
 - 인터럽트를 끄는 행위는 CPU 내부의 인터럽트 플래그(IF)를 끄는 행동으로, 인터럽트가 꺼지면 외부에서 인터럽트가 발생해도 CPU는 인터럽트의 발생을 체크하지 않는다. 한 프로그램이 인터럽트를 끄면 CPU는 인터럽트가 꺼진 상태로 계속 있게 되어, CPU가 다른 프로그램을 실행하더라도 인터럽트를 받을 수 없게 된다. CPU는 한 프로그램에게 독점될 수 없기 때문에 이 명령은 특권 명령으로 응용 프로그램이 실행할 수 없다.

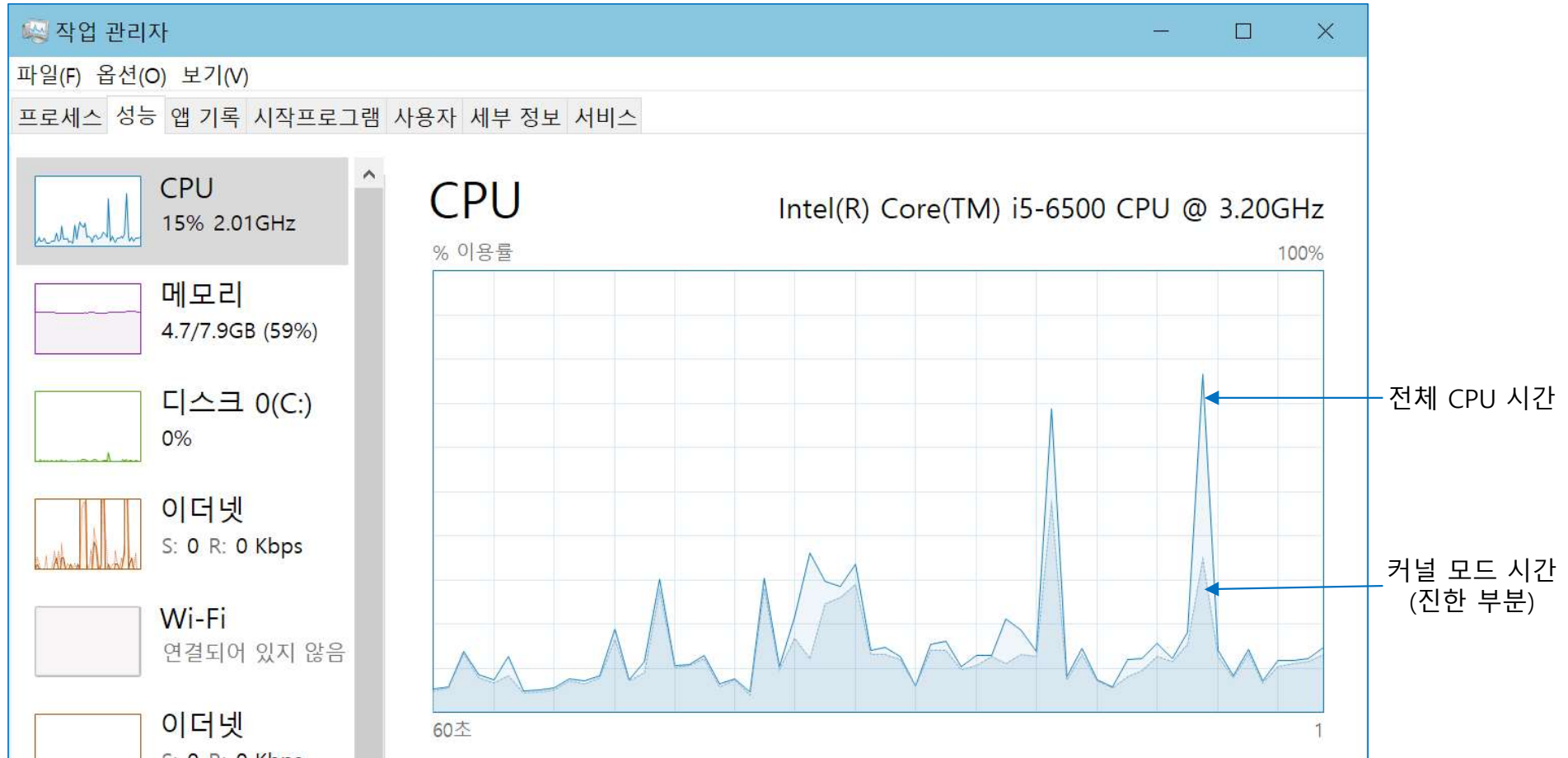
실행 모드와 관련된 다양한 이슈

51

- 1) 사용자 모드와 커널 모드는 CPU에 의해 구현되는가, 운영체제에 의해 구현되는가?
 - 모드는 CPU에 의해 구현되고 운영체제가 활용하는 기능
 - CPU 내부에 모드를 나타내는 레지스터 존재
 - 운영체제는 CPU 모드 레지스터를 이용하여 커널 영역 지킴
- 2) 운영체제가 사용자 모드와 커널 모드로 나누어 작동시키는 이유는?
 - 커널 공간(커널 코드와 데이터)에 대한 보안과 보호
 - 악의적 사용자와 오류 프로그램으로 커널 공간 지킴
 - 사용자 응용프로그램은 사용자 모드에서 아무리 심각한 오류가 발생해도 사용자 프로그램만 종료. 시스템을 중단시키지는 못함
- 3) 사용자 응용프로그램이 커널 코드를 호출하는 일이 있는가?
 - 사용자 응용프로그램을 직접 커널 코드 호출 불가
 - 시스템 호출을 통해서만 가능
- 4) CPU가 커널 모드와 사용자 모드 중 어떤 모드로 많이 실행될까?
 - 시스템 전체 통계를 보면 커널 모드에서 많이 실행
 - 키보드에서 읽고 디스플레이 출력, 디스크나 네트워크 작업 등 장치 액세스의 경우가 많으면 커널 모드 시간 비율 높음
 - 아무 작업도 없을 때 실행되는 시스템 유틸 프로세스가 커널 모드에서 실행되기 때문

Windows에서 전체 CPU 사용시간 중 커널 모드로 작동한 시간 보기

52



커널 모드의 시간 비율이 높은 대체적인 이유

- 아무 작업이 이루어지고 있지 않을 때, 커널 모드에서 실행되는 시스템 유틸 프로세스 때문

작업 관리자로 시스템 유틸 프로세스의 CPU 사용량 보기

53

작업 관리자

파일(F) 옵션(O) 보기(V)

프로세스 성능 앱 기록 시작프로그램 사용자 세부 정보 서비스

선택

이름	PID	상태	사용자 이름	CPU	메모리(활성...	UAC 가상화
시스템 유틸 시간 프...	0	실행 중	SYSTEM	97	8 K	
dwm.exe	1596	실행 중	DWM-1	01	114,116 K	사용 안 함
SnippingTool.exe	15212	실행 중	황기태	00	7,252 K	사용 안 함
System	4	실행 중	SYSTEM	00	20 K	
csrss.exe	836	실행 중	SYSTEM	00	1,620 K	허용 안 함
Taskmgr.exe	15620	실행 중	황기태	00	25,184 K	허용 안 함
atieclxx.exe	3508	실행 중	SYSTEM	00	2,108 K	허용 안 함
explorer.exe	7100	실행 중	황기태	00	137,856 K	사용 안 함
avp.exe	4900	실행 중	SYSTEM	00	71,848 K	허용 안 함
node.exe	11580	실행 중	황기태	00	38,084 K	사용 안 함
WUDFHost.exe	1380	실행 중	LOCAL SER...	00	1,468 K	허용 안 함
CoreSync.exe	12800	실행 중	황기태	00	147,424 K	사용 안 함
Adobe Desktop Serv...	9472	실행 중	황기태	00	80,416 K	사용 안 함
AdobePCBroker.exe	12124	실행 중	황기태	00	5,576 K	사용 안 함
node.exe	12960	실행 중	황기태	00	29,480 K	사용 안 함

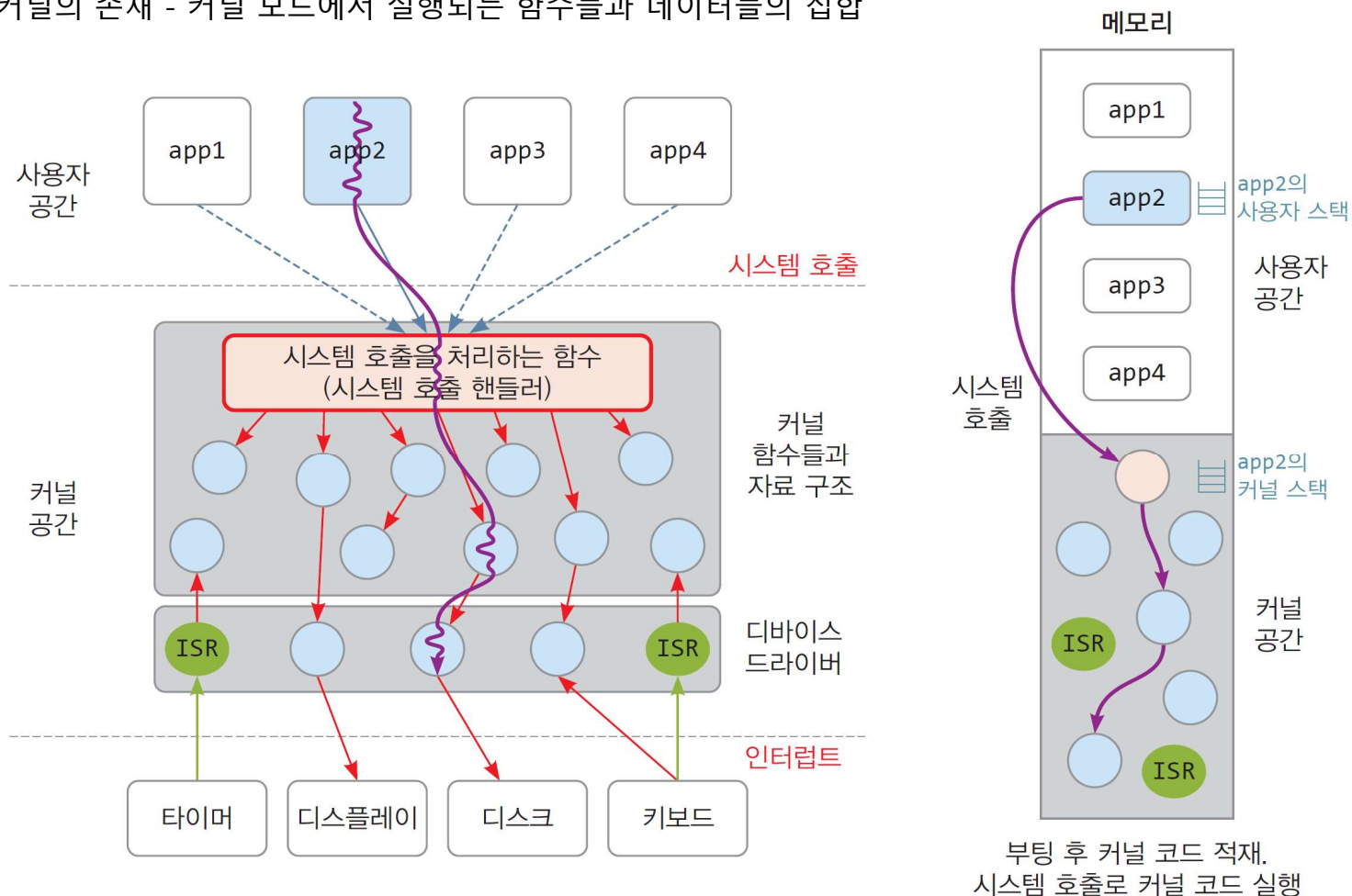
간단히(D)

작업 끝내기(E)

커널의 실체(1)

54

- 커널은 부팅 시에 커널 공간에 적재된 함수들과 데이터 집합
 - ▣ 커널은 컴파일된 바이너리 형태, 하드디스크 특정 영역에 저장, 부팅 시에 커널 공간의 메모리에 적재
- 커널 코드는 함수들의 집합
 - ▣ 커널의 존재 - 커널 모드에서 실행되는 함수들과 데이터들의 집합



* 현재 커널 코드를 실행하고 있는 것은 누구인가? app2 프로세스이다. 커널 프로세스란 말은 없다. 커널은 프로세스가 아니다.

커널의 실체(2)

55

- 커널은 스스로 실행되는 프로세스인가? NO
 - ▣ 커널은 함수들의 단순 집합, 시스템 호출을 통해 호출되는 함수들
 - ▣ 커널이 스케줄링한다(x)
 - 커널 프로세스가 실행되면서 주기적으로 스케줄링한다(x)
 - 시스템 호출과 인터럽트 서비스 루틴에 의해 커널 내 스케줄러 함수가 호출되어 실행(0)
- 커널은 실행 중이다? NO
 - ▣ 커널은 프로세스도 스레드도 아니므로 NO
 - ▣ 커널이 실행 중이다(x)
 - 응용프로그램이 시스템 호출을 하여 커널 코드를 실행하고 있다(0)
 - 인터럽트가 발생하여 인터럽트 서비스 루틴이 실행되고 있다(0)
- 커널은 스택이나 힙을 가지는가? NO
 - ▣ 커널은 스택이나 힙을 가지는 주체가 아니다. 그러므로 NO
 - ▣ 스택이나 힙을 가지는 주체는 프로세스나 스레드
 - 교재 P.75, P.134, P.205 참고
 - ▣ 스레드마다 사용자 스택과 커널 스택 소유
 - 스레드가 생성될 때 프로세스의 사용자 공간에 사용자 스택 할당
 - 사용자 코드가 실행되는 동안 활용
 - 스레드가 생성될 때 커널 공간에 커널 스택 할당
 - 스레드가 시스템 호출로 커널 코드를 실행할 때 스택으로 활용

응용프로그램 빌딩

56

- 라이브러리(library)
 - ▣ 응용프로그램에서 활용하도록 미리 작성된 함수들, 컴파일되어 바이너리 형태로 제공되는 파일
 - ▣ 개발자는 라이브러리 활용없이 응용프로그램 작성 불가능

- 응용프로그램이 활용하는 라이브러리는 2가지 유형
 - ▣ **표준 라이브러리(Standard Library)**
 - 사용자가 작성하기 힘든 함수 제공
 - 운영체제나 컴퓨터 하드웨어에 상관없이 이름과 사용법 동일
 - 운영체제나 하드웨어, 컴파일러에 관계없이 호환
 - ▣ **시스템 호출 라이브러리(System Call Library)**
 - 시스템 호출 함수들 포함
 - 시스템 호출 함수들은 시스템 호출을 진행하여 커널 모드로 바꾸고 커널로 진입하여 커널에 만들어진 함수 실행(커널의 다양한 기능 수행)
 - 운영체제마다 시스템 호출 함수의 이름이 서로 다름
 - 운영체제 비호환
 - 시스템 호출 함수를 커널 API(Application Programming Interface)라고 부름

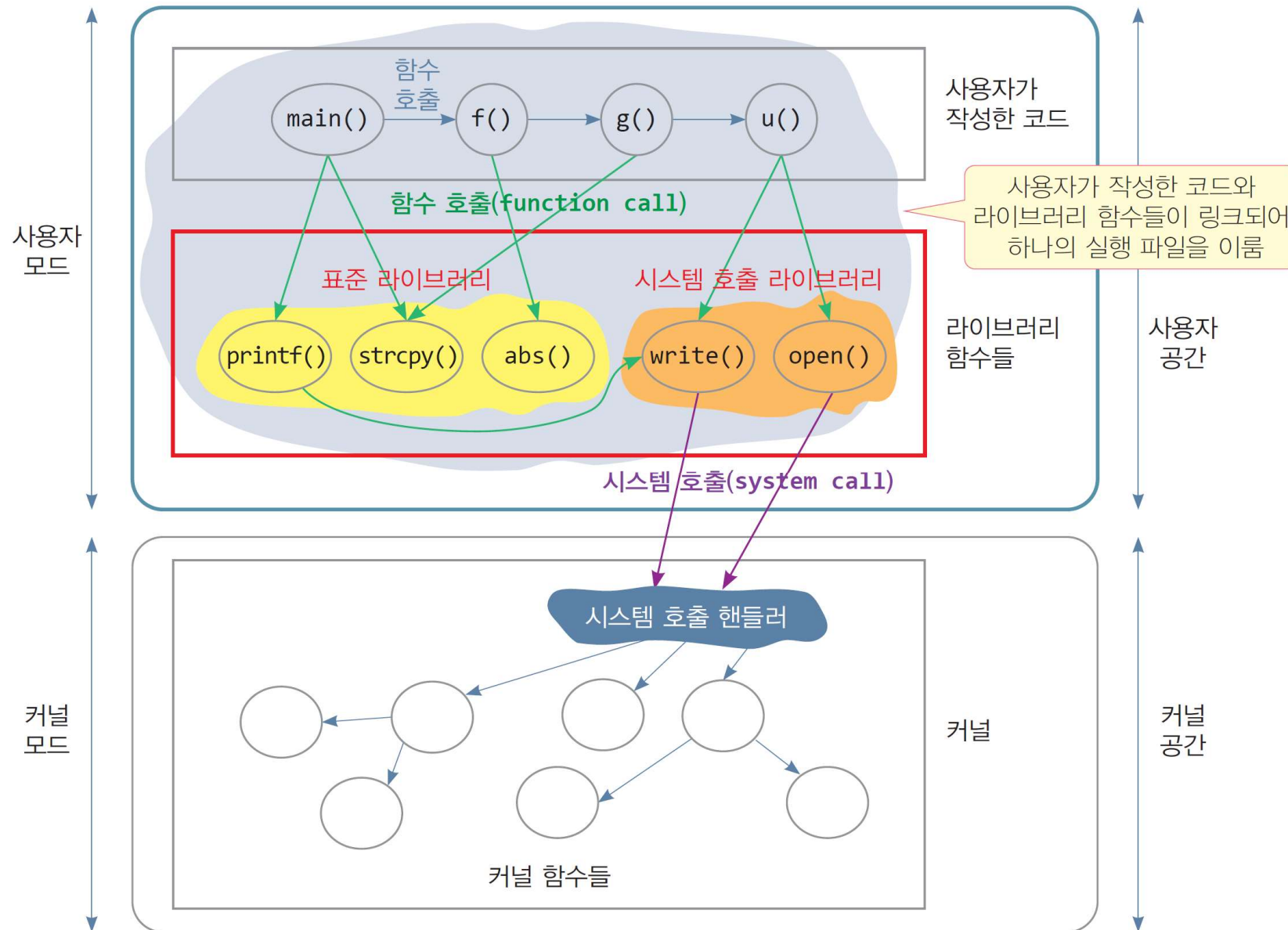
사용자 코드와 라이브러리 코드의 링킹

57

- 실행 파일이 만들어지는 과정
 - ▣ 응용프로그램 코드는 라이브러리 코드와의 링킹을 거쳐 하나의 실행 파일로 만들어짐
- 응용프로그램 실행
 - ▣ 응용프로그램이 사용자 공간에 적재
 - 실행 파일 내 사용자 코드와 라이브러리 코드의 메모리 적재
 - 실행 파일 내 사용자 전역 변수와 라이브러리의 전역 변수 모두 메모리 적재
 - ▣ 응용프로그램은 사용자 모드로 실행 시작
 - ▣ 사례) 다음 슬라이드

응용프로그램을 구성하는 사용자 코드와 라이브러리, 그리고 커널

58



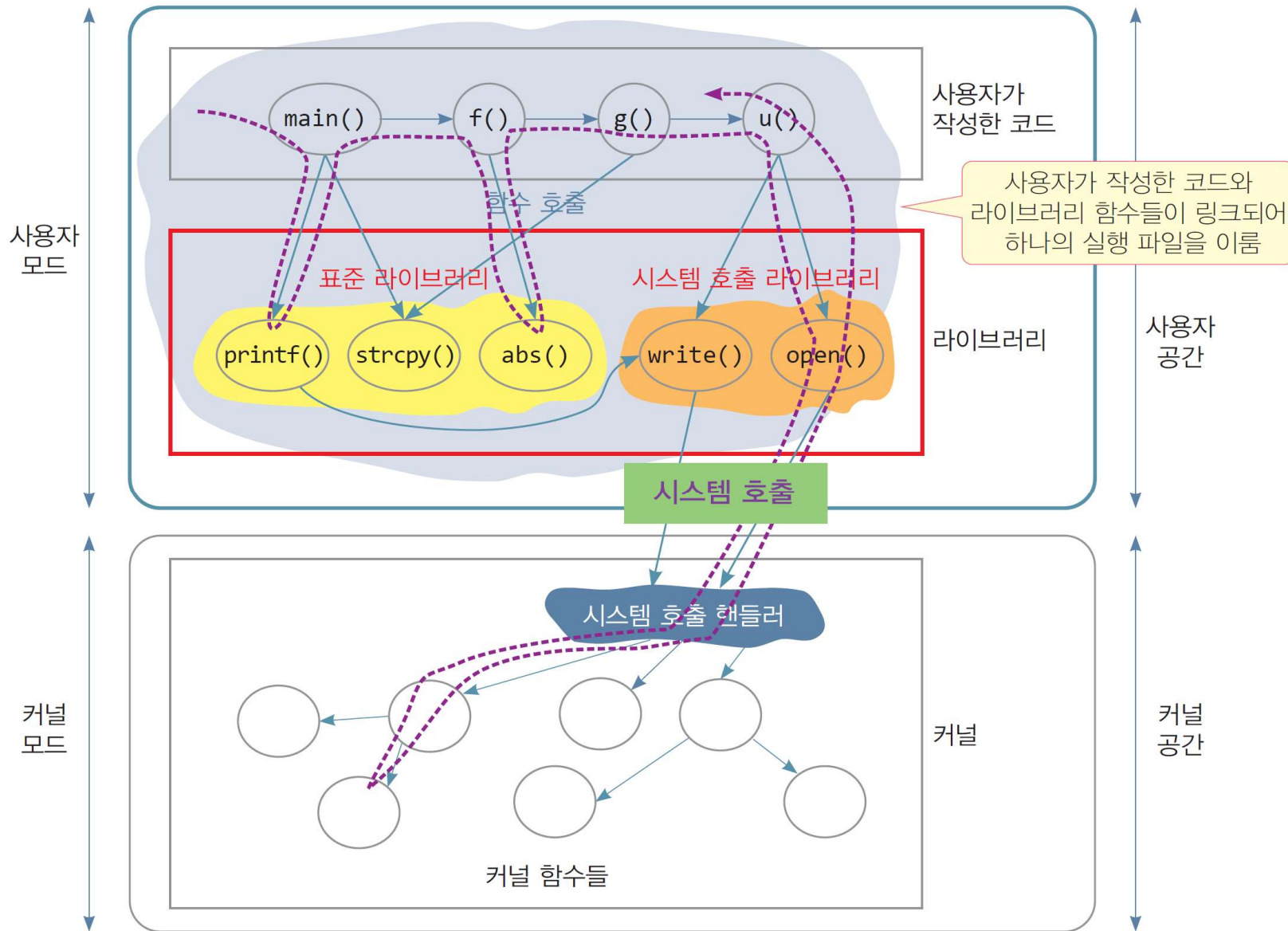
함수 호출과 시스템 호출

59

- 함수 호출(function call)로 라이브러리 활용
 - 사용자 공간에 적재된 함수가 사용자 공간에 적재된 다른 함수(표준 라이브러리나 시스템 호출 라이브러리의 함수 포함)호출
 - 사용자 공간에서, 사용자 모드에서 실행
- ▣ 함수 호출 과정
 - 사용자 공간의 스택에 돌아올 주소, 매개변수 전달, 호출된 함수의 지역변수 생성
 - 사용자 공간에 적재된 함수의 주소로 점프
 - 함수가 끝나면 함수를 호출한 곳으로 복귀
- 시스템 호출(system call)로 커널 코드 실행
 - 응용프로그램이 운영체제의 기능을 사용하고자 할 때, 커널에 작성된 함수 실행
 - 시스템 호출 라이브러리에 포함된 시스템 호출 함수가 시스템 호출 일으킴
- ▣ 시스템 호출 과정
 - 시스템 호출을 일으키는 특별한 기계 명령 실행
 - 이 명령이 사용자 모드에서 커널 모드로 전환, 커널 함수마다 매겨진 고유 번호 전달
 - 커널의 시스템 호출 핸들러 실행
 - 시스템 호출 핸들러가 전달받은 커널 함수의 고유 번호 분석, 해당 커널 함수 호출
 - 커널 함수에서가 리턴할 때 사용자 모드로 전환, 사용자 프로그램으로 복귀

응용프로그램에서 라이브러리와 커널 코드 호출 사례

60



*커널 함수들은 이름이 없고, id 번호로 구분된다.

시스템 호출

61

□ 시스템 호출

- 사용자 공간의 코드에서 커널 서비스를 요청하는 과정
 - 사용자 공간의 코드가 커널 함수를 호출하는 과정
 - 커널 콜(kernel call), 트랩(trap) 로도 불림
 - 응용프로그램에서 커널 기능을 활용하도록 만들어 놓은 기능
- 운영체제는 시스템 호출 라이브러리 제공
 - 시스템 호출 함수 혹은 커널 API 포함
 - Unix/Linux의 커널 API – open(), read(), write(), fork(), exit()
 - Windows의 커널 API – CreateProcess(), WaitForSingleObject()
 - 대략 200개 이상의 시스템 호출 함수 있음

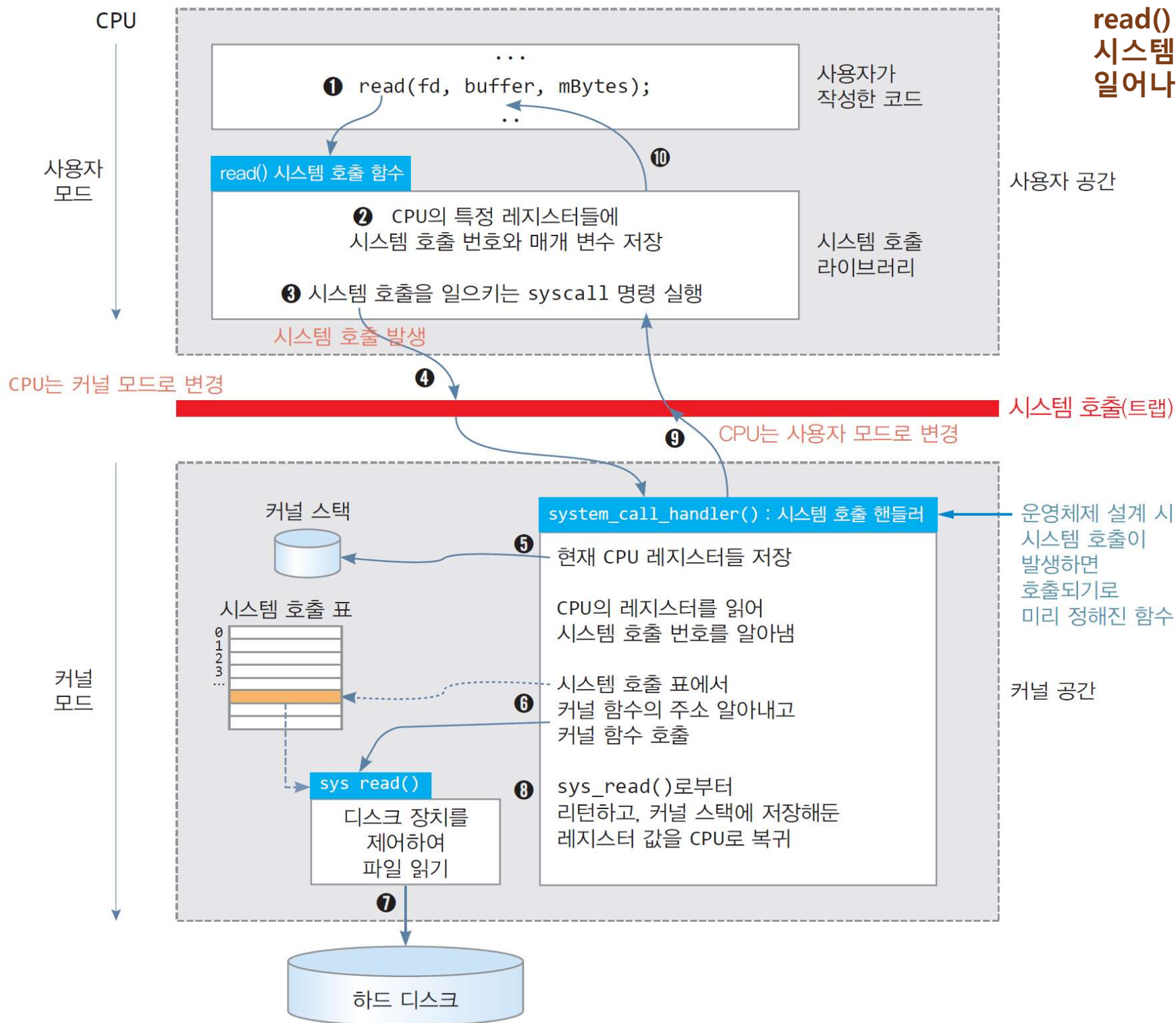
□ 시스템 호출을 일으키는 기계 명령

- CPU마다 시스템 호출을 실행하는 특별한 기계 명령 제공
 - 시스템 호출 CPU 명령
- 사례
 - int 0x80 - 인텔의 x86계열의 CPU, 32비트에서 사용
 - syscall/sysret - ADM에서 최초 구현. 64비트에서만 작동
 - sysenter/sysexit - Intel에서 최초 구현, X86/64 CPU, AMD

□ 표준 라이브러리를 통해 간접적으로 이루어지는 시스템 호출

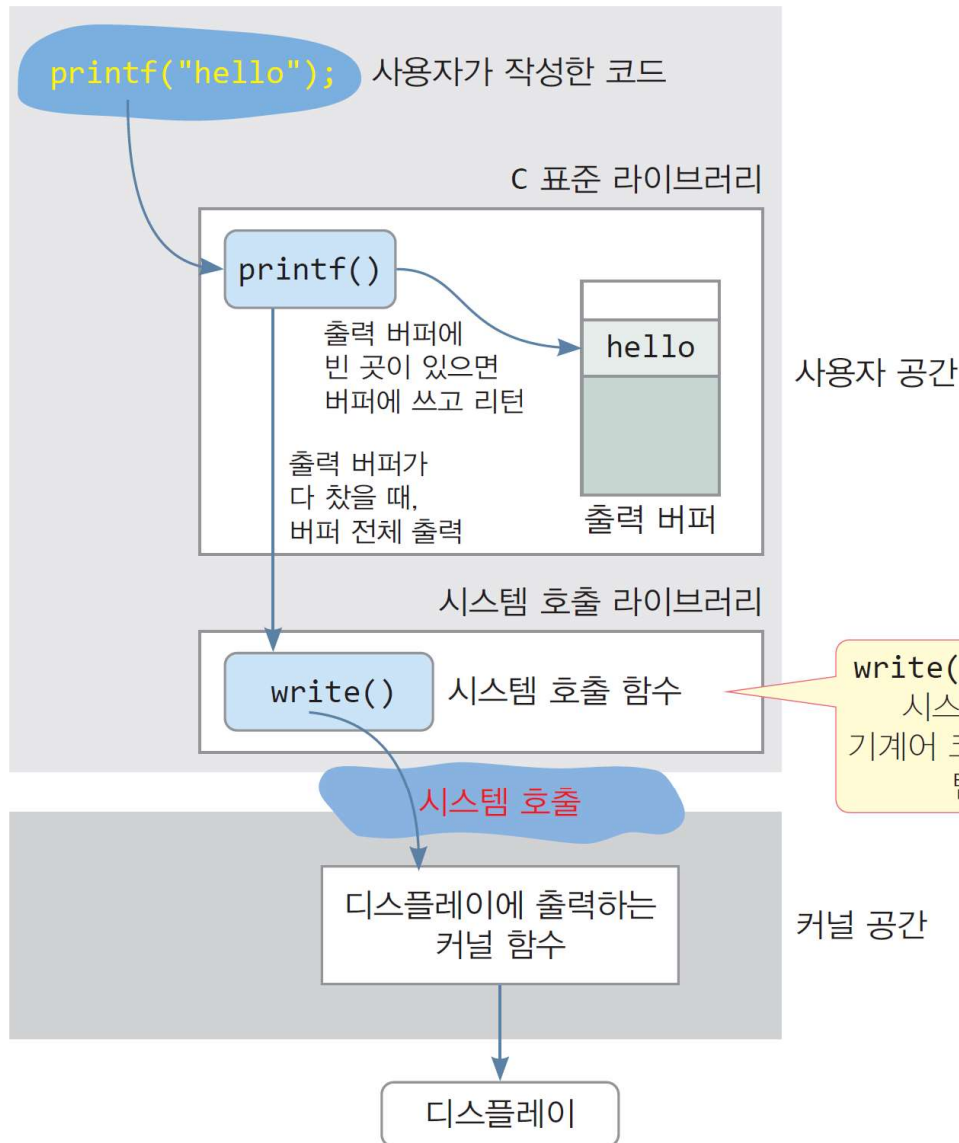
- 응용프로그램 -> 시스템 호출 라이브러리의 시스템 호출 함수 -> 시스템 호출 CPU 명령
- 응용프로그램 -> 표준 라이브러리 함수 -> 시스템 호출 라이브러리의 시스템 호출 함수 -> 시스템 호출 CPU 명령

read() 함수에 의해 시스템 호출이 일어나는 과정



탐구 2-1 printf()가 직접 디스플레이에 출력할까?

63



printf() 함수는 표준 라이브러리 함수이다.
printf() 함수를 호출하면 시스템 호출이 일어날까?

- 디스플레이에 접근하는 것은 커널만 가능
- printf() 함수는 디스플레이에 직접 출력 불가능
- printf()는 C 표준 라이브러리의 버퍼에 출력
- 버퍼가 꽉 차 있으면, printf()는 시스템 호출 함수 write() 호출
- write() 함수는 '시스템 호출 CPU 명령' 실행
- 커널에 작성된 함수가 디스플레이에 "hello" 출력

write() 시스템 호출 함수에서
시스템 호출을 일으키는
기계어 코드는 어떻게 생겼을까?
탐구 2-2를 보자.

탐구 2-2 syscall 기계 명령으로 write 시스템 호출 코드 작성

64

리눅스에서 어셈블리어를 이용하여 직접 syscall 기계 명령으로 시스템 호출을 일으키는 사례이다.

```
$ gcc -c hello.s && ld hello.o
$ ./a.out
Hello world
$
```

hello.s

```
# 화면에 "Hello, World"를 출력하는 어셈블리어 프로그램으로 hello.s에 저장
# 64-bit 리눅스에서 실행
#
# 프로그램 시작
.global _start
.text
_start:
    # write(1, msg, 12) 시스템 호출, 화면 출력
    mov $1, %rax          # 1은 write 시스템 호출 번호, rax 레지스터에 저장
    mov $1, %rdi          # 1은 표준출력장치(디스플레이)의 파일 디스크립터 번호
    mov $msg, %rsi        # msg는 출력할 문자열의 주소
    mov $12, %rdx         # 12는 출력할 바이트 수
    syscall               # 시스템 호출 기계 명령
                          # 화면에 Hello world를 출력하는 write 시스템 호출 실행

    # exit(0) 시스템 호출, 프로그램 종료
    mov $60, %rax         # 60은 exit 시스템 호출 번호
    xor %rdi, %rdi        # xor 명령 후 rdi 레지스터 값은 0이 됨. 0은 종료 코드
    syscall               # 시스템 호출 기계 명령
                          # 프로그램을 종료시키는 exit 시스템 호출 실행

msg:
    .ascii "Hello world\n"
```

프로그램 실행 시작

화면 출력을 요청하는 시스템 호출

프로그램 종료를 요청하는 시스템 호출

시스템 호출 비용 : fread()와 read()의 비교

65

- 시스템 호출은 함수 호출에 비해 많은 시간 비용
 - ▣ 시스템 호출을 많이 할수록 프로그램 실행 속도 저하
- 파일에서 1000바이트를 읽는 2가지 유형의 코드. 실행 비교 결과는?

```
char buf[100];  
...  
for(i=1; i<=10; i++)  
    fread(fp, buf, 100);
```

C 표준 함수 fread() 사용.
1번의 시스템 호출. 속도 빠름

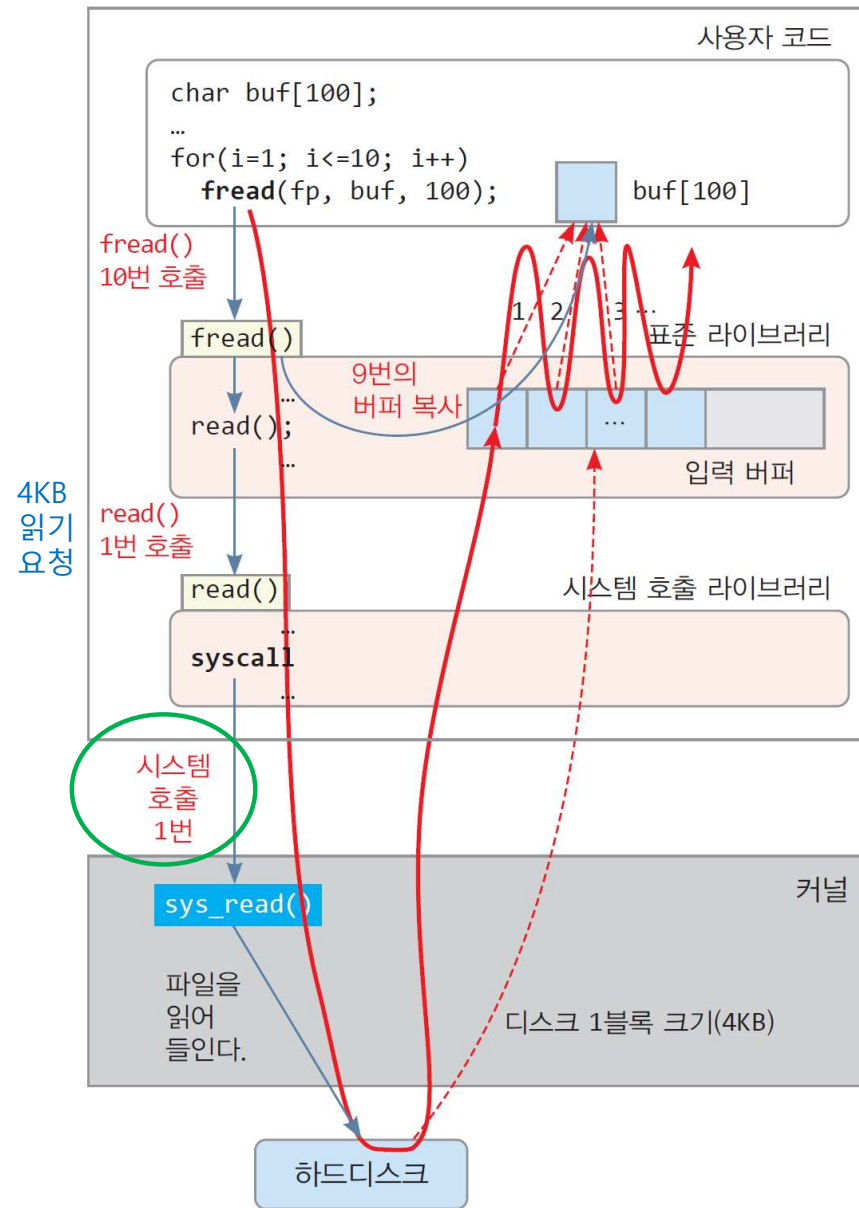
VS.

```
char buf[100];  
...  
for(i=1; i<=10; i++)  
    read(fd, buf, 100);
```

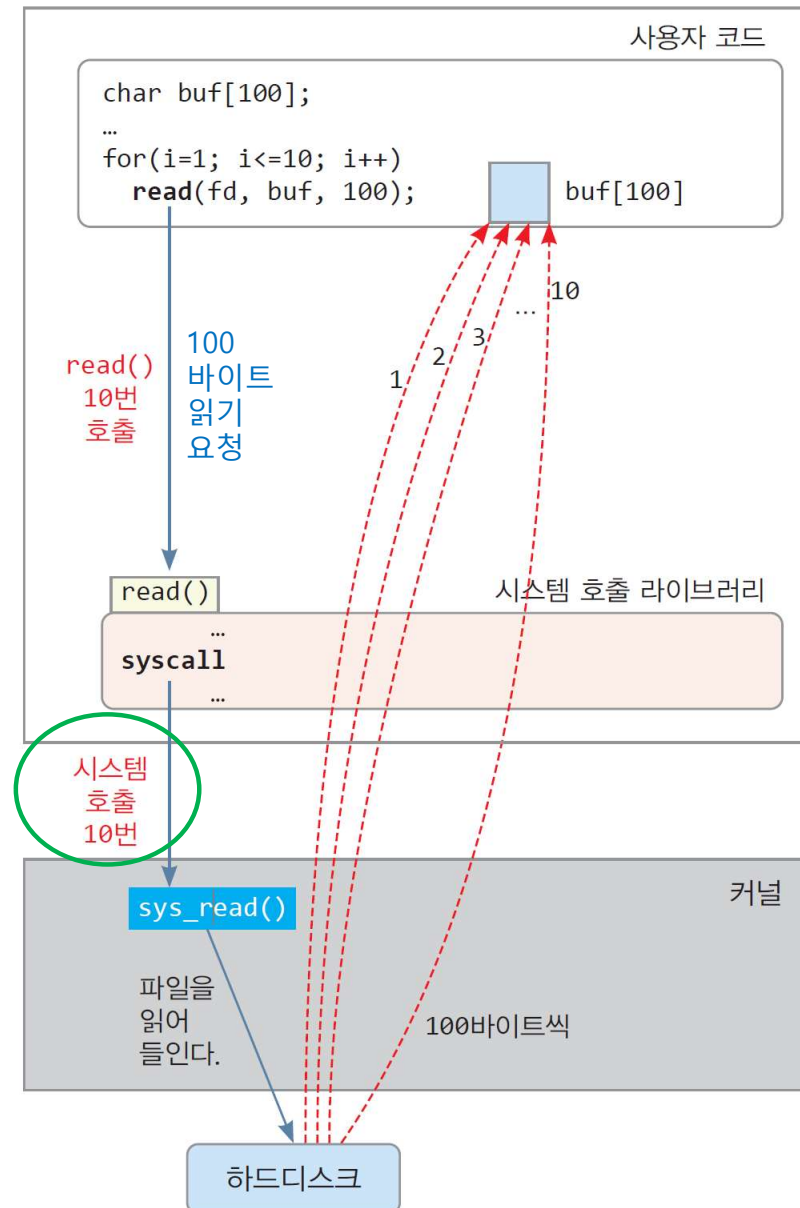
시스템 호출 함수 read() 사용.
10번의 시스템 호출. 느리고 비효율적

- 표준 C 라이브러리 함수 fread()와 시스템 호출 read() 실행 비교
 - ▣ 표준 C 라이브러리 함수, fread(fp, buf, size)의 동작 과정
 - fread()를 처음 호출하면 라이브러리 내 버퍼(buffer)가 비어 있음
 - read()를 호출하여 라이브러리 내 버퍼를 채운다. n = read(fd, buf, BLKSIZE);
 - 응용프로그램으로부터 요청 받은 크기(size)만큼 응용프로그램의 buf로 복사
 - 라이브러리 버퍼가 비거나 부족하면 그때 다시 read() 호출
 - ▣ 시스템 호출 함수, read(fd, buf, size)의 동작 과정
 - 시스템 호출을 이용하여 커널 코드 실행
 - 커널 코드에서 디스크 읽기
 - 라이브러리를 거치지 않고 바로 buf로 읽어 들임

fread와 read()의 동작 과정 비교



(a) 표준 라이브러리 함수 fread()를 사용한 경우

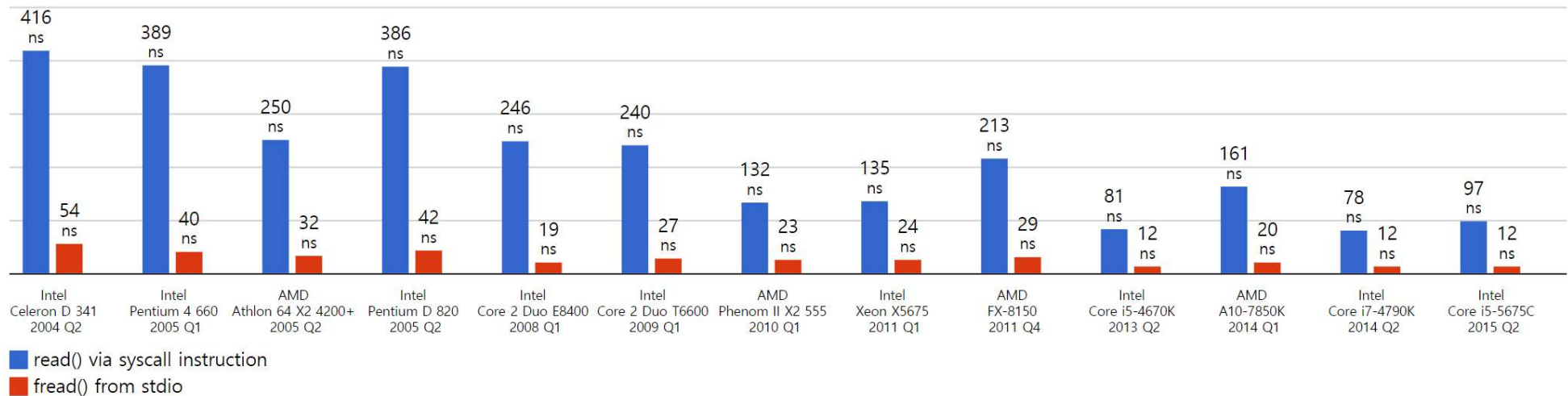


(b) 시스템 호출 함수 read()를 사용한 경우

fread()와 read()의 시간 측정 결과 비교

67

read()와 fread()의 실행 시간 결과



(출처 : <http://arkanis.de/weblog/2017-01-05-measurements-of-system-call-performance-and-overhead>의 그래프를 저자가 축소)

결과 분석 : fread()가 read()에 비해 약 8배 빠름

시스템 호출에 따른 비용 정리

68

□ 시스템 호출에 따른 비용은 매우 크다

시스템 호출 함수에 전달할 데이터들을 CPU 레지스터에 저장

-> 사용자 모드에서 커널 모드로 전환

-> 시스템 호출 핸들러 실행

-> CPU 내 레지스터가 훼손되는 것을 막기 위해 스택에 저장

-> 시스템 호출 표에서 시스템 호출 함수 찾기

-> 시스템 호출 함수 실행

-> 스택에 저장해둔 레지스터 복귀

-> 사용자 모드로 복귀

□ 시스템 호출은 필연적이지만, 시스템 호출 횟수를 줄여야 응용프로그램의 실행 시간이 짧아지고, 시스템 입장에서 더 많은 프로그램을 실행시킬 수 있는 시간 확보 -> 시스템의 처리율 향상

	함수 호출	시스템 호출
메모리 영역	사용자 영역의 코드에서 사용자 영역의 함수 호출	사용자 영역의 코드에서 커널 함수 호출
CPU 실행 모드	사용자 모드	사용자 모드에서 커널 모드로 전환
비용	함수 호출에 따른 비용	커널 모드로 전환하는 등 함수 호출에 비해 큰 비용

4. 운영체제와 인터럽트

인터럽트

70

□ 인터럽트

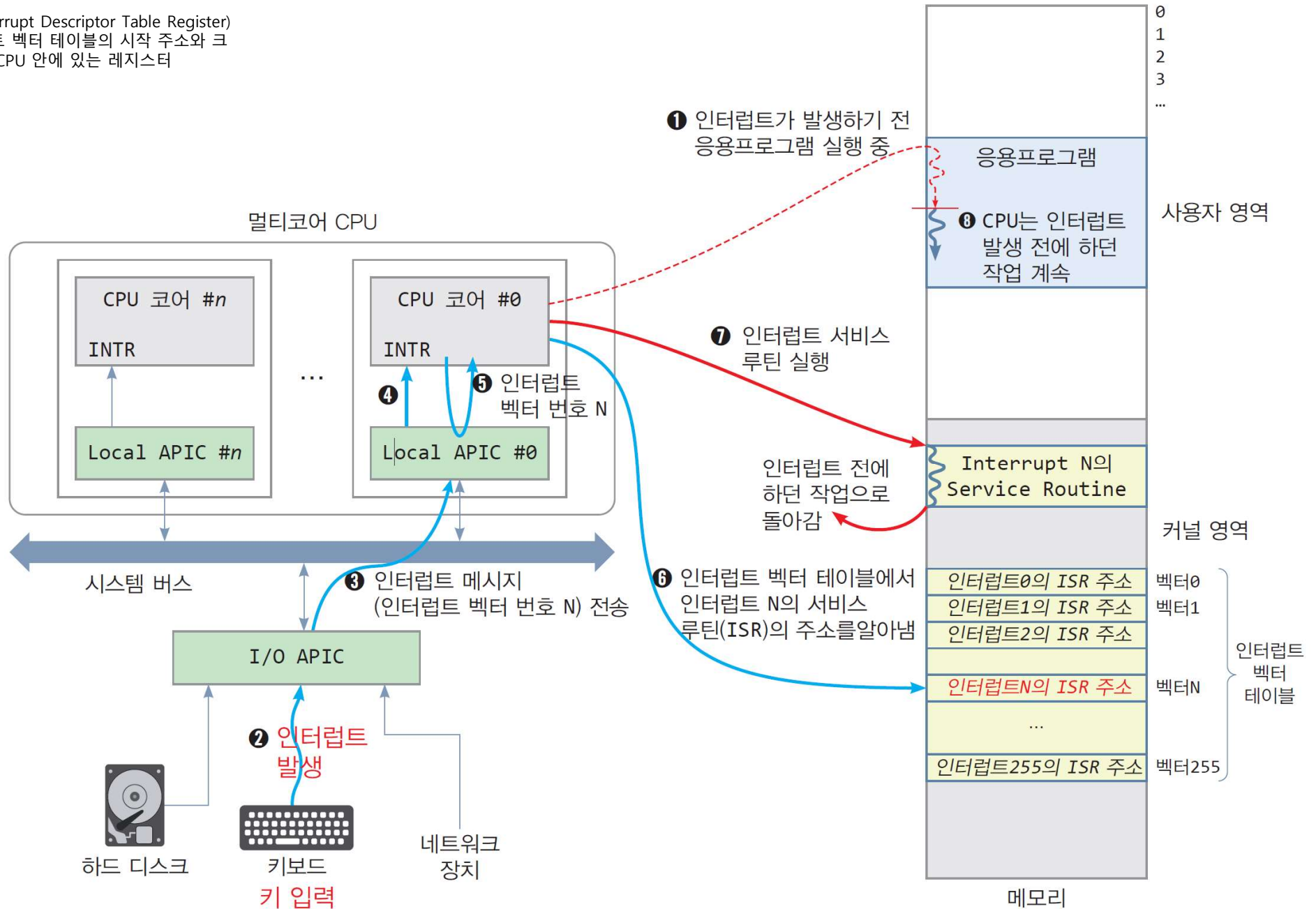
- 입출력 장치들이 비동기적 사건을 CPU에게 알리는 행위
 - 비동기적이란 예정되지 않거나 발생시간을 예측할 수 없는 사건
 - (키보드 입력, 네트워크로부터 데이터 도착 등)
- 하드웨어 인터럽트
 - 장치들이 어떤 상황 발생을 CPU에게 알리는 하드웨어 신호
 - CPU는 인터럽트를 수신하면 인터럽트 서비스 루틴 실행
- 소프트웨어 인터럽트
 - CPU 명령으로 발생시키는 인터럽트
 - 하드웨어 인터럽트를 수신한 것과 동일하게 처리

□ 컴퓨터에서 인터럽트 활용

- 마우스를 움직이거나 클릭하는 등 마우스 조작
- 키보드 입력
- 네트워크로부터 데이터 도착
- 하드디스크의 쓰기 종료
- 시스템 클럭으로부터 일정한 시간 간격으로 알림
- 컴퓨터의 리셋 버튼 누르기
- USB 메모리 부착 혹은 해제

인터럽트 발생 및 처리과정

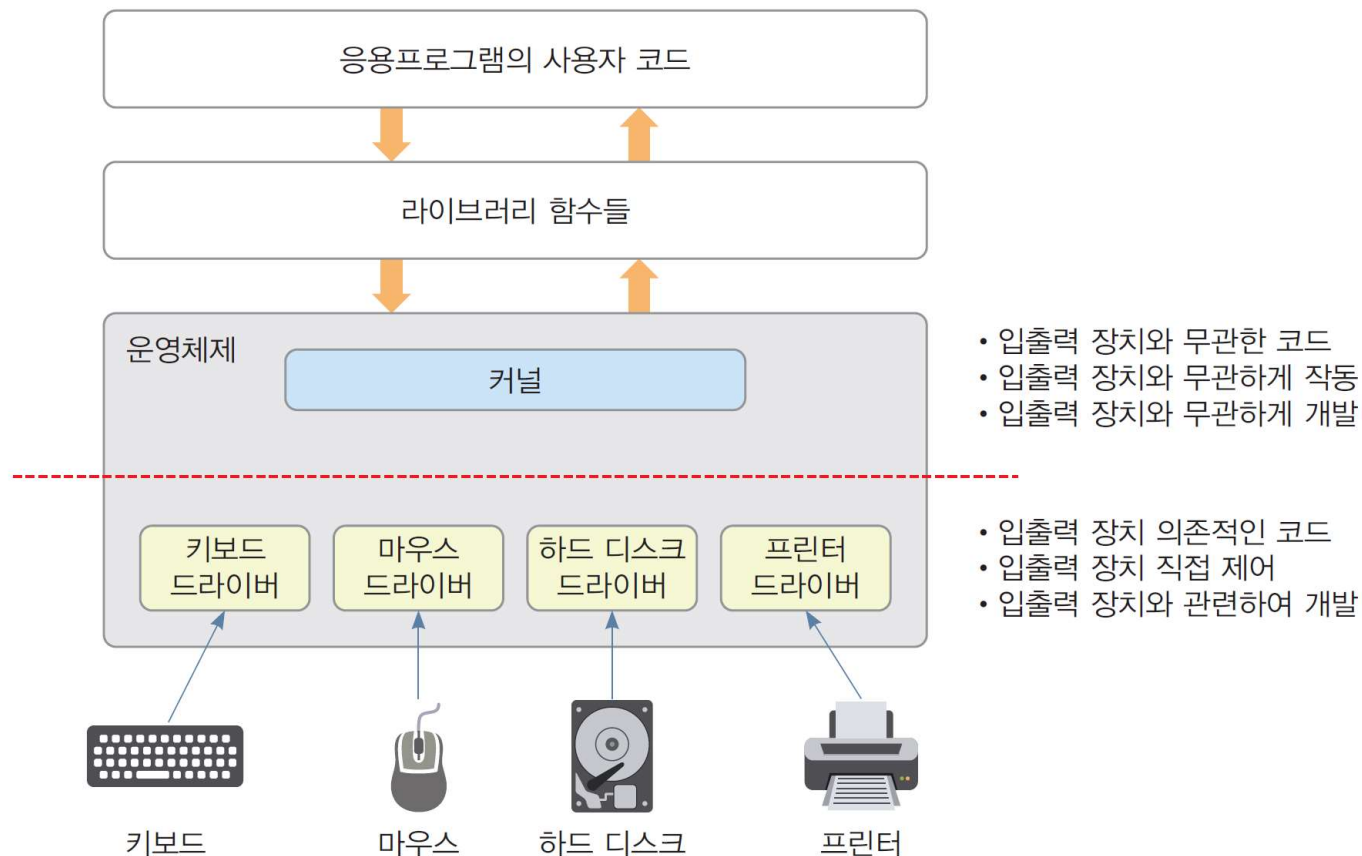
* IDTR(Interrupt Descriptor Table Register)
은 인터럽트 벡터 테이블의 시작 주소와 크
기를 가진 CPU 안에 있는 레지스터



인터럽트 서비스 루틴과 운영체제

72

- 인터럽트 서비스 루틴
 - ▣ Interrupt Service Routine, ISR, 인터럽트 핸들러
 - ▣ 위치 : 디바이스 드라이버나 커널 코드, 임베디드 컴퓨터 ROM
- 디바이스 드라이버와 인터럽트 서비스 루틴, 그리고 커널



인터럽트는 다중프로그래밍 실현의 키

73

- 다중프로그래밍 리뷰
 - ▣ 여러 프로세스를 동시에 실행
 - ▣ 한 프로세스가 입출력을 시행하면 다른 프로세스로 교체 실행
 - ▣ 입출력이 완료될 때, 장치로부터 입출력 완료 통보를 받는 방법 필요 -> 그것이 바로 **인터럽트**
 - 인터럽트가 없다면, CPU는 입출력 완료를 계속 검사하는 폴링 (polling)을 실행해야 하므로 매우 비효율적
- 인터럽트가 없다면
 - ▣ 다중프로그래밍 운영체제의 구현은 사실상 거의 불가능
- 인터럽트의 효과
 - ▣ 입출력 장치와 CPU가 동시에 각자의 작업 실행
 - 입출력 장치는 지시받은 입출력 진행, CPU는 다른 프로그램 실행
 - ▣ 컴퓨터 시스템이 효율적으로 작동
 - CPU 활용률이 높아지고, 시스템 처리율 향상